UART Controlled Digital System

Detailed architecture, module functionality, command behavior, and verification review

This project implements a serial command processor. An external master sends UART frames that request register access or an ALU operation. The receiver reconstructs each byte, the system controller interprets the command, and the transmitter returns register data or a two-byte ALU result.

The implementation separates fast processing on REF_CLK from serial communication derived from UART_CLK. A receive data synchronizer transfers accepted bytes into the processing domain. An asynchronous FIFO buffers response bytes and transfers them into the transmit domain.

Implementation fact	Default value
Processing clock	50 MHz in specification and testbench
UART source clock	3.6864 MHz in specification and testbench
UART payload and register width	8 bits
ALU result width	16 bits
Register file	16 entries; addresses 0x0 through 0xF
Response FIFO	8 entries of 8 bits
Reset	Active low; asynchronous assertion with synchronized release at the two root clocks

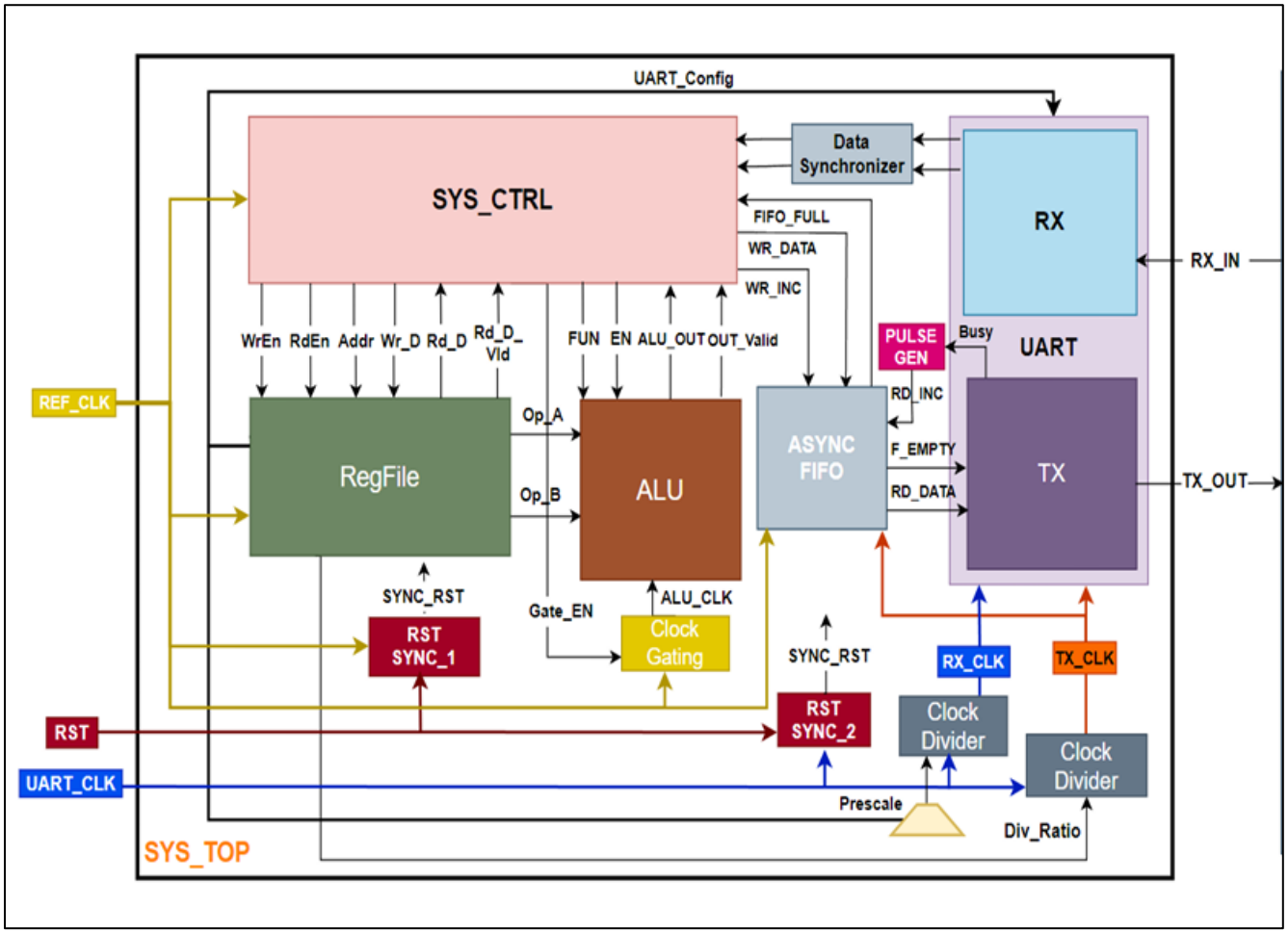
Basis and scope

This document is based on the three supplied specification PDFs, two supplied UART diagrams, and all 33 supplied source or source-list files. RTL behavior takes precedence where a specification is ambiguous. The figures are redrawn from the actual module connections. No RTL was modified.

Verification status: RTL review plus simulation evidence supplied in Saeed_Maher_System.pdf. That report records successful checks for all seven test scenarios. The results summary and waveform figures appear on pages 28-34. No new HDL simulation was run for this documentation update; reported results are attributed to the supplied run.

1. System overview

1.1 System Architecture



The receive direction is UART_RX_IN to UART_RX, then DATA_SYNC, then SYS_CTRL. Accepted bytes advance a command parser. The controller writes operands into the register file or requests a register read. ALU operands are continuously exposed from register locations 0 and 1.

The response direction is SYS_CTRL to the FIFO write interface, then the FIFO head to UART_TX. Register reads enqueue one byte. ALU operations enqueue the latched result as low byte followed by high byte. UART_TX begins a frame whenever it is idle and the FIFO is nonempty.

UART parity and stop errors are exposed as top-level status outputs. They do not form an error response packet. Clock dividers and reset synchronizers support the data path but are omitted from this figure to keep the command path clear.

1.2 Top level interface and hierarchy

Port	Direction	Meaning
RST_N	Input	Global active-low reset
REF_CLK	Input	Processing reference clock
UART_CLK	Input	UART source clock before division
UART_RX_IN	Input	Serial command stream, idle high
UART_TX_OUT	Output	Serial response stream, idle high
parity_error	Output	Receiver parity-error status during error handling
stop_error	Output	Receiver stop-bit-error status during stop handling

Integration responsibilities

SYS_TOP in system_top.v instantiates the controller, ALU, register file, clock gate, two clock dividers, two reset synchronizers, receive data synchronizer, response FIFO, pulse generator, and UART wrapper. UART_TOP contains independent TX and RX submodules and shares the parity configuration between them.

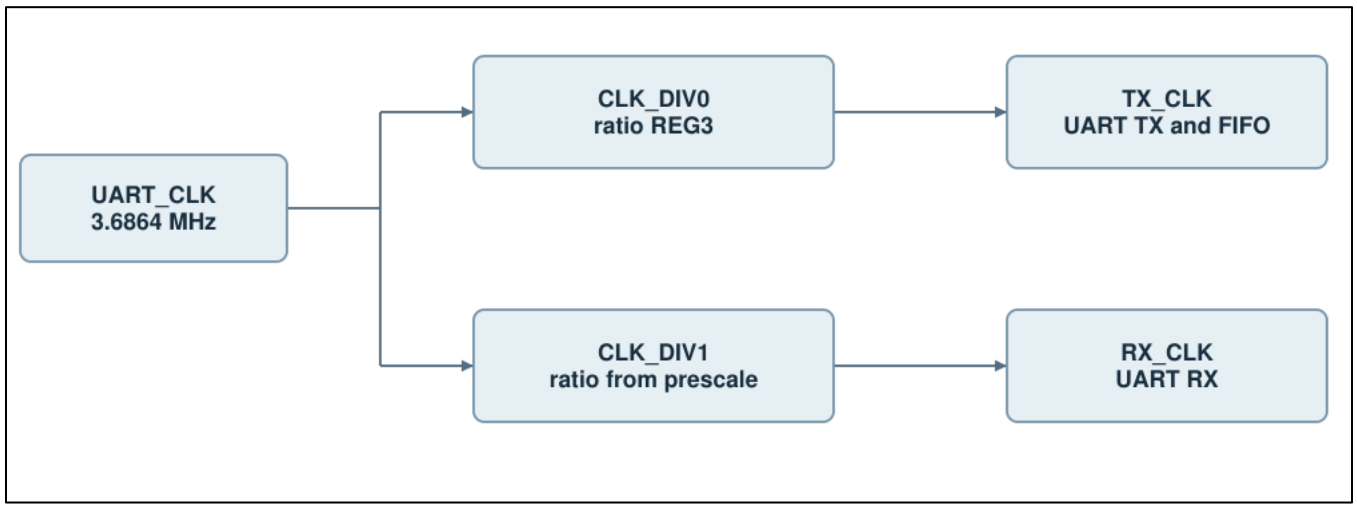
The controller runs on REF_CLK. Register operations also run on REF_CLK. ALU registers run on the gated form of REF_CLK. The FIFO writes on REF_CLK and reads on TX_CLK. UART_RX runs on RX_CLK, which is generated by the second divider.

Default parameter relationships

DATA_WIDTH = 8, RF_ADDR = 4, ALU_OUTPUT_WIDTH = 16, ALU_FUNC_WIDTH = 4, DATA_SYNC_STAGES = 2, and RST_SYNC_STAGES = 2. The instantiated register file depth remains fixed at 16 and the FIFO depth remains fixed at 8.

The result-splitting logic expects each half of ALU_OUTPUT_WIDTH to fit one UART byte. For this integration, ALU_OUTPUT_WIDTH = 2 x DATA_WIDTH. A different parameter set needs review of hard-coded bit indices, terminal counts, and port widths before use.

1.3 Clock generation and baud rate



The transmit divider uses REG3 as its ratio D. For ordinary divided operation, f_TX = f_UART / D. One TX clock period represents one serial bit. At the default D = 32, the nominal baud rate is 115200 bit/s and one bit lasts approximately 8.68056 microseconds.

RX prescale P	RX divider Q	RX clock	Bit rate f_RX / P
32	1 bypass	3.6864 MHz	115200 bit/s
16	2	1.8432 MHz	115200 bit/s
8	4	0.9216 MHz	115200 bit/s

The top level chooses Q = 4 for P = 8, Q = 2 for P = 16, and Q = 1 otherwise. Thus all three supported RX prescales retain the same nominal receive baud rate at the stated source frequency. Matching transmit and receive baud requires D = P x Q, which equals 32 for these settings.

Important: the shorter BIT_TIME_16 and BIT_TIME_8 constants in the testbench describe a different timing assumption. They are unused by the executed tests and should not be reused without accounting for CLK_DIV1.

1.4 Configuration clock gate and reset

Register or field	Reset value	Role
REG0 / address 0x0	0x00	ALU operand A
REG1 / address 0x1	0x00	ALU operand B
REG2[0]	1	Parity enable
REG2[1]	0	Parity type: 0 even, 1 odd
REG2[7:2]	32	RX oversampling prescale
REG3[7:0]	32	TX clock division ratio

REG2 resets to decimal 129, or 0x81: six-bit prescale 32, even parity, and parity enabled. REG3 resets to 0x20. Configuration outputs are continuously connected to the UART and divider logic; there is no separate commit or synchronization handshake for these settings.

Clock divider behavior

Integer_CLK_DIV reloads an eight-bit down-counter and toggles a clock register at count zero and at half the ratio. Even ratios have equal-length high and low phases; odd ratios have unequal phases. A ratio of zero or one, or a low i_clk_en, selects the original reference clock at the output. Disable therefore means bypass, not clock stop. The internal counter continues running.

Clock gating

CLK_gate captures clk_en while REF_CLK is low and ANDs the stored enable with REF_CLK. Holding the enable during the high phase avoids truncating a high clock pulse in the intended latch-based RTL model. SYS_CTRL opens the ALU clock in ALU_FUN, ALU_EXECUTION, and WRITE_IN_FIFO, and closes it in other states. A commented library-cell implementation is not active.

Reset release

RST_SYNC asynchronously clears its pipeline on reset assertion, then shifts in ones on root-clock rising edges. Two stages release REF_SYNC_RST_n after two REF_CLK edges and UART_SYNC_RST_n after two UART_CLK edges. The latter also feeds TX_CLK and RX_CLK logic; release is not separately synchronized to each divided clock. NUM_STAGES must be at least two with the current slice expressions.

1.5UART command and response protocol

Command	Payload bytes after command	Response
0xAA register write	Address, data	None
0xBB register read	Address	One byte from register file
0xCC ALU with operands	Operand A, operand B, function	Two bytes: result[7:0], then result[15:8]
0xDD ALU with stored operands	Function	Two bytes: result[7:0], then result[15:8]

Each table entry represents a sequence of UART payload bytes, not a single UART frame containing the entire command. All payload bytes travel least significant bit first within their individual UART frames. Parity, when enabled, applies independently to every byte.

Command examples

```
AA 05 55       Write 0x55 into register address 0x5
BB 05          Read address 0x5; response payload 55
CC 0C 03 00    Compute 12 + 3; response payloads 0F 00
DD 02          Multiply stored 12 and 3; response 24 00
```

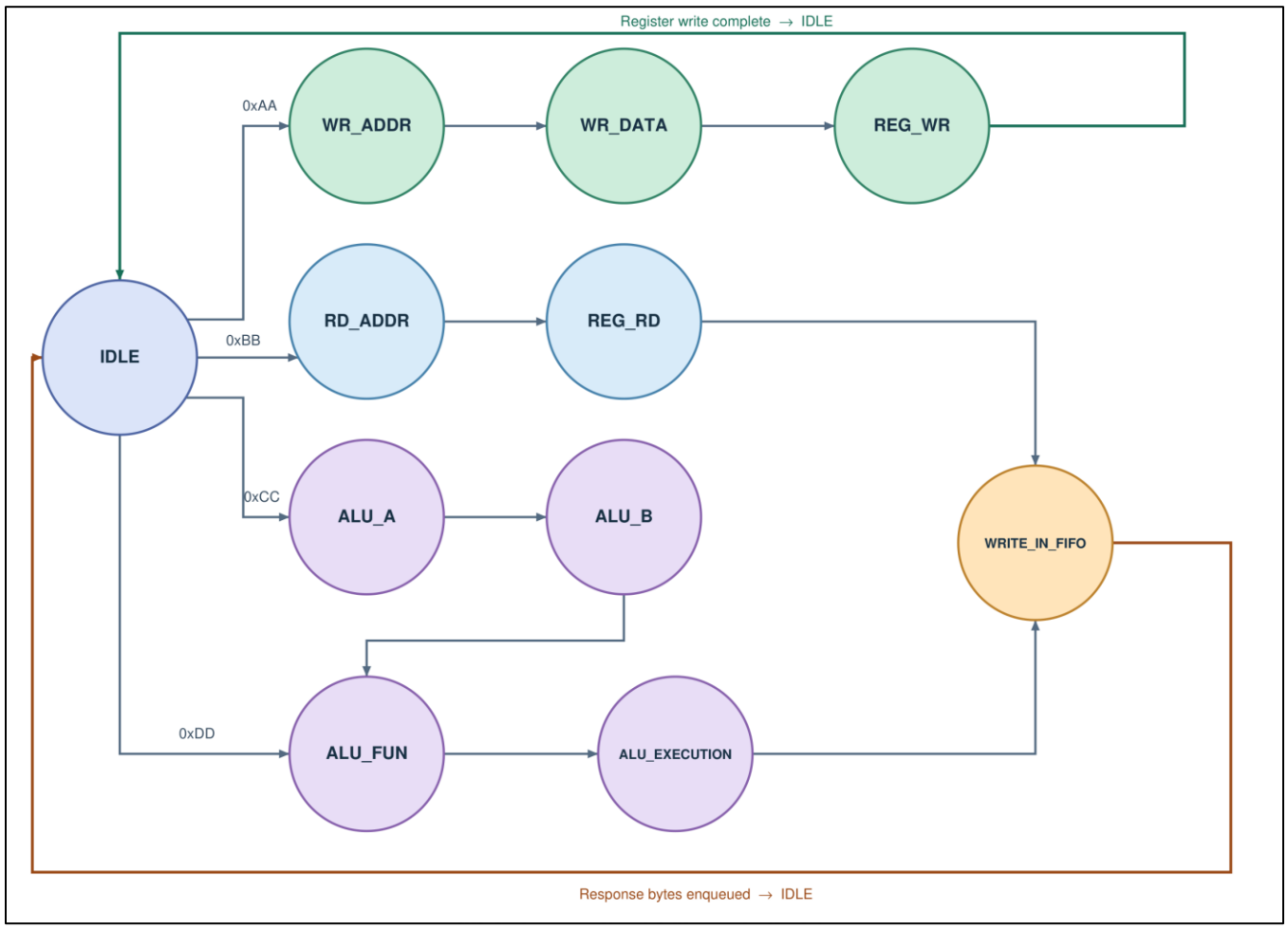
The examples use hexadecimal byte notation. The upper nibble of an address byte is ignored because the controller selects rx_data[3:0]. The function byte likewise uses its lower four bits. No checksum, packet length field, command identifier in the response, or write acknowledgement is implemented.

Master behavior

The parser handles one command at a time. There is no receive command queue or ready output to stop the master. The master must keep frame settings consistent, complete each command in order, and avoid sending another command while the controller cannot accept it. Waiting for a response is useful for read and ALU operations; writes need sufficient processing time before a dependent action.

Unknown command bytes in IDLE are ignored. A missing payload byte leaves the parser waiting in its current collection state. A corrupt payload frame is suppressed by UART_RX and can therefore leave a partially collected command waiting indefinitely; no command timeout or abort is implemented.

1.6 System controller state machine



Payload collection transitions occur only when rx_data_valid is asserted. REG_WR returns to IDLE after one controller cycle. REG_RD waits for RD_valid. ALU_EXECUTION waits for ALU_out_valid. WRITE_IN_FIFO returns to IDLE after one accepted read-response byte or two accepted ALU-response bytes. Return arrows and self-loops are omitted from the drawing.

The controller is a Moore-style state-driven control block with selected inputs used directly in output expressions. addr_reg retains the write address. op_sel distinguishes register-read responses from ALU responses. ALU_out_lat stores the completed result, and ALU_part selects its low or high half.

1.7 Controller transaction sequencing

Register write

IDLE recognizes 0xAA. WR_ADDR waits for and captures the low address bits in addr_reg. WR_DATA waits for the data byte. REG_WR then asserts WR_en_regfile for one cycle and presents the held receive bus as write data. The receive synchronizer holds its output between accepted bytes, so the data remains available for this write edge.

Register read

IDLE recognizes 0xBB. RD_ADDR waits for the address byte. REG_RD asserts RD_en_regfile and uses the held receive bus for the address. The register file updates RdData and rd_valid at a REF_CLK edge. The controller then enters WRITE_IN_FIFO and offers that byte to the FIFO. There is no response header.

ALU operation

For 0xCC, ALU_A writes the first operand into REG0 and ALU_B writes the second into REG1, each gated by rx_data_valid. For 0xDD, these states are skipped, preserving the stored operands. Both commands converge at ALU_FUN, which presents the low function bits and pulses ALU_en when the function byte arrives.

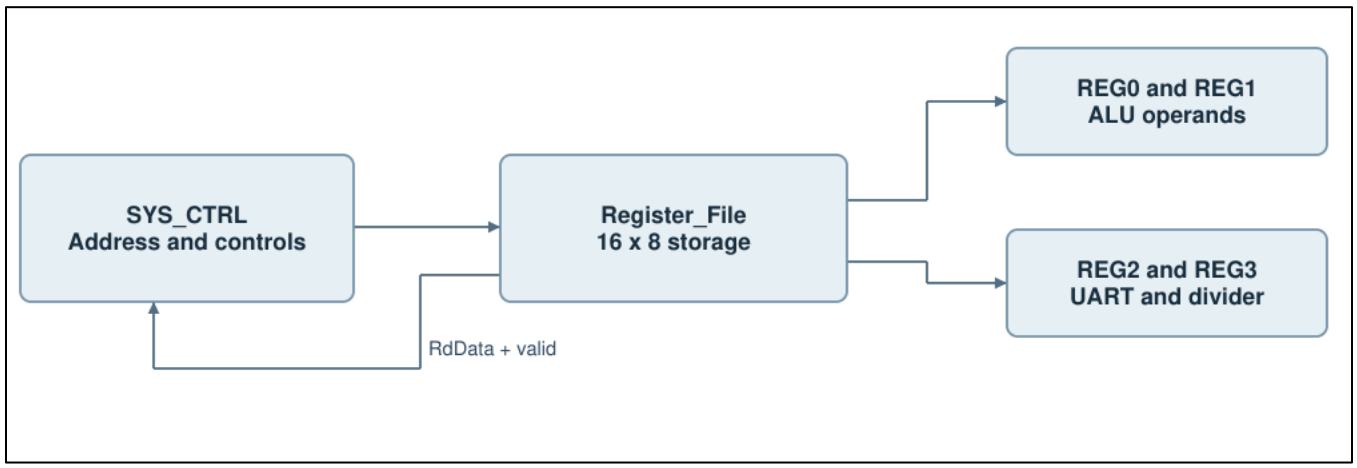
The clock gate is already open while waiting in ALU_FUN. At the sampling edge, the ALU registers its result and valid flag. ALU_EXECUTION observes the valid result and latches it into ALU_out_lat before response packing begins. The latched copy is necessary because the ALU drives zero with valid low on later enabled clock edges when enable is low.

Response packing and FIFO backpressure

In WRITE_IN_FIFO, WR_inc_fifo remains high. The FIFO internally blocks writes while full. The controller also stays in this state while full and changes ALU_part only on a non-full cycle. For an ALU result, ALU_part = 0 selects the low byte and the next accepted cycle selects the high byte. After that cycle, the selector returns to zero and the controller returns to IDLE.

This backpressure protects response storage, but it does not protect the receive command path: incoming command bytes arriving during an execution or response state have no general queue. Sustained command rates therefore need system-level testing.

1.8 Register file functionality



Register_File contains 16 byte-wide registers. A rising CLK edge performs a write when WrEn is high and RdEn is low. It performs a read when RdEn is high and WrEn is low. If both enables are high, neither operation occurs. With both low, stored data and the previous RdData are retained.

rd_valid is cleared by default each active edge and set on a read. If RdEn stays asserted across several cycles, valid stays high across those read cycles. The controller uses this handshake to determine when the response byte can be enqueued.

Address range	Function	Access behavior
0x0	Operand A	Continuously exposed as op_A
0x1	Operand B	Continuously exposed as op_B
0x2	UART configuration	Continuously exposed as UART_CON
0x3	TX divider ratio	Continuously exposed as DIV_ratio
0x4 through 0xF	General storage	Normal synchronous reads and writes

All addresses are writable; there is no hardware protection for reserved registers. The specification mentions a normal range ending at 0x15, but that is incompatible with the actual four-bit address bus and depth 16. The implemented upper address is 0xF. Values such as 0x15 are truncated to address 0x5 by the controller.

1.9 ALU functions

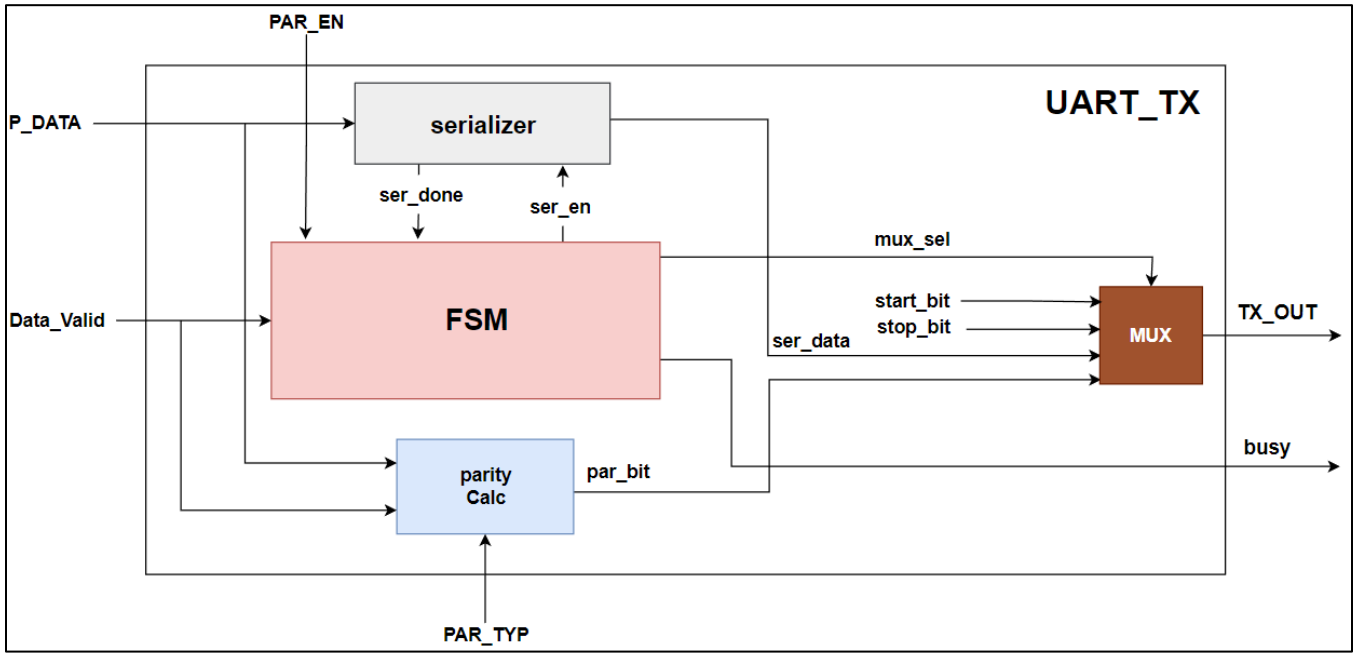
ALU takes unsigned eight-bit operands and a four-bit function. Combinational logic computes a 16-bit result; ALU_OUT and out_valid are registered on the gated clock. The following encodings are taken directly from ALU.v and rand_pkg.sv.

Code	Operation	Result behavior
0x0	Addition	A + B
0x1	Subtraction	A - B in the 16-bit result context
0x2	Multiplication	A x B; full 8 by 8 product
0x3	Division	Unsigned quotient; B = 0 gives 0xFFFF
0x4 / 0x5	AND / OR	A & B / A | B
0x6 / 0x7	NAND / NOR	~(A & B) / ~(A | B)
0x8 / 0x9	XOR / XNOR	A ^ B / A ~^ B
0xA	Equal	1 when A = B, otherwise 0
0xB	Greater than	2 when A > B, otherwise 0
0xC	Less than	3 when A < B, otherwise 0
0xD / 0xE	Shift right / left	A >> 1 / A << 1
0xF	Unsupported function	Result 0; out_valid = 0

The expressions are assigned directly into a 16-bit destination. In particular, complement operations are not explicitly masked back to eight bits. Their upper result bits follow Verilog expression-sizing rules; they should not be documented as guaranteed zero. The supplied run shows upper ones for complemented results, including XNOR = 0xFF2B. A left shift retains a ninth bit in the wider result context, as shown by 250 shifted left returning 0x01F4.

There are no carry, negative, zero, or overflow output flags in this ALU. When enable is low, the next clocked outputs become zero and invalid. An unsupported function creates no valid result, so SYS_CTRL remains in ALU_EXECUTION until reset because it has no timeout or invalid-operation response.

1.10 UART transmitter architecture



UART_TX converts a parallel byte into a framed serial output. FSM_TX selects a constant one for idle and stop, a constant zero for start, serialized data for eight bit intervals, and the computed parity bit when parity is enabled.

P_data_reg captures P_data when data_valid is high and ser_en is low. This is not an idle-only condition: it can also capture during parity or stop. The serializer has completed the current byte by then, so the current data bits no longer depend on the register. Integrations should still respect the module timing rather than assume a conventional ready/valid acceptance port.

Parity_gen_TX registers reduction XOR for even parity or reduction XNOR for odd parity whenever data_valid is high. In this system data_valid is the FIFO nonempty level, not a one-cycle strobe. That integration detail lets parity update after P_data_reg captures the byte. A standalone one-cycle data_valid pulse needs checking because the parity register sees the previous P_data_reg on the capture edge.

Transmitter frame sequencing

State	TX output	Action and next state
IDLE	1	busy = 0; valid selects START
START	0	busy = 1; serializer begins; next DATA
DATA	Serialized bit	Wait for ser_done; then PARITY or STOP
PARITY	Parity bit	One TX clock interval; next STOP
STOP	1	One TX clock interval; next IDLE

Serializer

parallel_2_serial_TX selects data[counter] and registers it while en is high. The counter starts at zero and increments under the same enable. This transmits the least significant bit first. The START interval prepares the first registered serial bit for the DATA state.

done is registered from the condition counter == 7. The FSM uses done to stop serialization and enter parity or stop. The current counter width is derived from WIDTH, but the terminal comparison is hard-coded to seven. Correct generalization beyond eight bits needs more than changing the parameter.

Frame timing

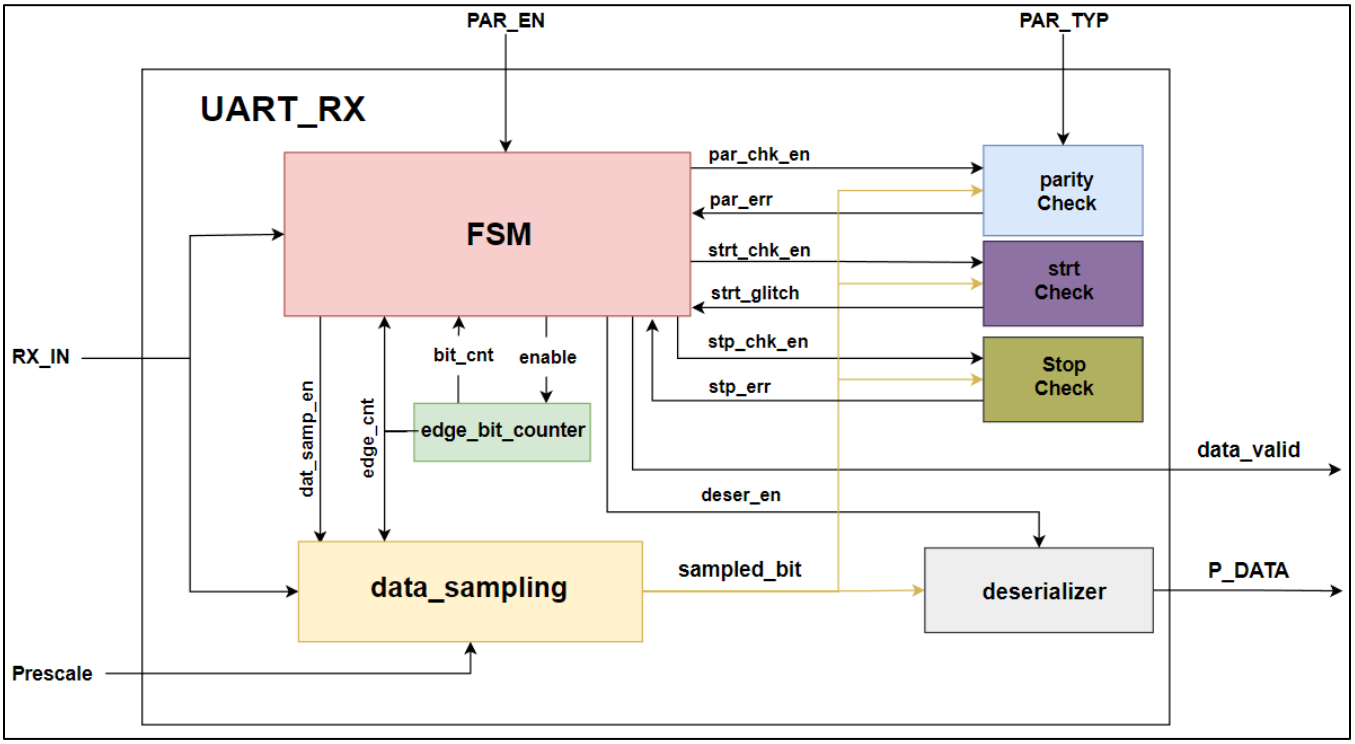
A parity-enabled frame contains 11 serial bits: one start, eight data, one parity, and one stop. Without parity it contains 10 bits. At 115200 bit/s those durations are approximately 95.49 microseconds and 86.81 microseconds. The FSM passes through IDLE between successive frames, adding an idle clock interval when more data is already queued.

Busy and response consumption

busy is high in every state except IDLE. Its rising edge signals that a FIFO head byte has been accepted into the transmit sequence. Pulse_generator converts this transition into a pulse that advances the FIFO read pointer on a TX_CLK edge. The FIFO can then expose the next byte while the current byte is serialized from P_data_reg.

There is no packet-level busy output at SYS_TOP. The external master sees UART_TX_OUT and the two receive error signals. Response boundaries are determined by command type: one byte for a read and two bytes for an ALU command.

1.11 UART receiver architecture



UART_RX detects a low line as a candidate start. The sampler takes three observations around the middle of each bit and produces a majority decision. The counter tracks sampling-clock edges within a bit and the bit position within a frame. The FSM enables the appropriate checker and the deserializer.

The deserializer shifts only when the sampled-bit-valid signal and serial_en are both asserted. Start, parity, and stop bits are checked rather than shifted into the payload. P_data reflects the shift register throughout reception; DATA_SYNC captures it only after a valid frame indication.

The receiver supports prescale values 8, 16, and 32. It does not contain a dedicated input synchronizer for RX_IN before the FSM and sampler. Oversampling improves bit decisions but is not a replacement for implementation-level metastability analysis.

Receiver sampling and validation

Sampling and bit reconstruction

For prescale P, sampler_RX captures RX_IN at edge_count = P/2 - 1, P/2, and P/2 + 1. It asserts valid_bit when taking the third sample. Sequential consumers observe the updated sample and strobe on the following clock edge. The majority function chooses the value present in at least two of the three stored samples.

Serial_2_parallel_RX shifts {bit, shift_reg[7:1]} for each accepted payload bit. With least-significant-bit-first reception, eight shifts reconstruct the byte in its normal bit order. This shift slice is hard-coded to eight-bit operation.

Frame checks and error visibility

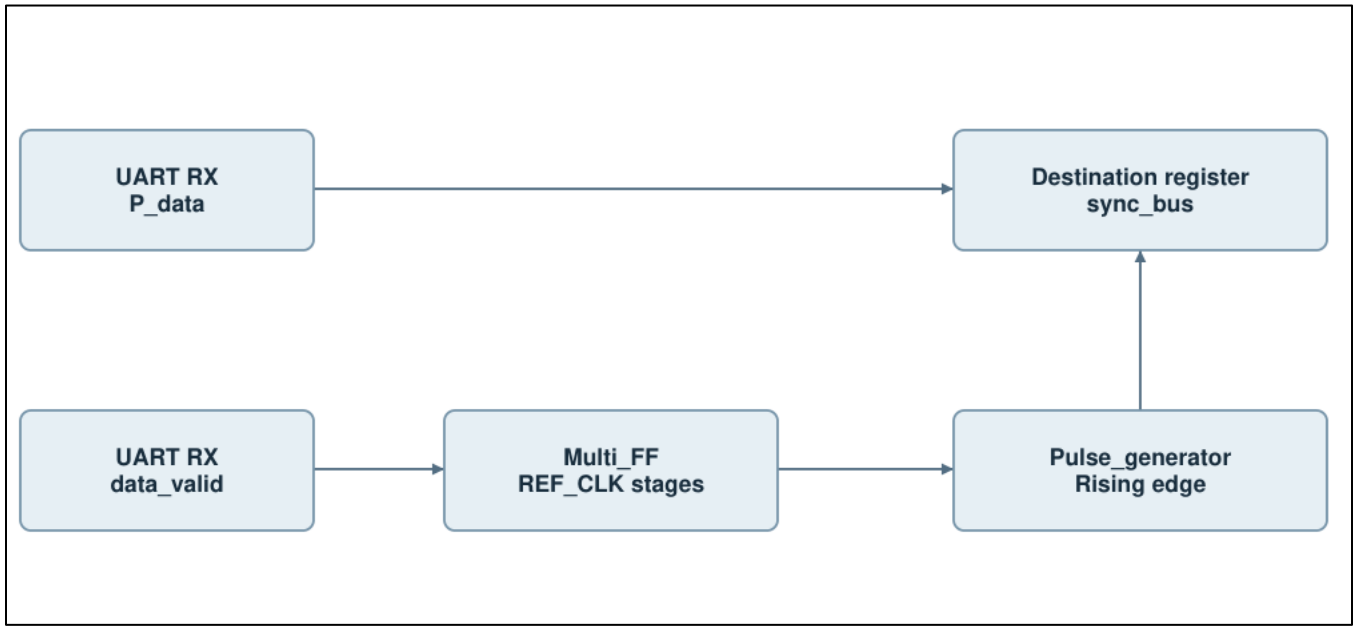
Checker	Enabled during	Meaning
Start_check_RX	START_CHK	A sampled one indicates a false start or glitch
Parity_check_RX	NORMAL_DATA and PARTIY_CHK	One-bit accumulator toggles for each sampled one, including parity
Stop_check_RX	STOP_CHK or PAR_ERR_STOP_CHK	A sampled zero indicates a stop error

For even parity, the accumulator should end at zero; for odd parity, it should end at one. The parity FSM branch leads to PAR_ERR_STOP_CHK on failure. That branch consumes the stop interval but does not publish valid data. Stop checking is also performed on the normal STOP_CHK path.

The FSM sets data_valid near the end of STOP_CHK when edge_count equals P - 2, and only if stop_error is low. The register holds its value during other STOP_CHK cycles, so the code does not guarantee an exactly one-clock pulse despite its comment. The rising-edge pulse in DATA_SYNC reduces the synchronized valid level to one destination event.

Parity and stop errors are state-dependent outputs rather than sticky status registers. At frame end, the FSM can go directly to START_CHK if RX_IN is low, supporting consecutive frames. Counter recovery after a rejected start should be tested because edge_count and bit_count are not explicitly cleared on every return to IDLE.

Receive data synchronization



DATA_SYNC uses a multi-cycle-path style bus transfer: it synchronizes the control level rather than placing independent synchronizer chains on every data bit. Multi_FF shifts bus_enable through NUMBER_STAGES flip-flops in REF_CLK. A rising-edge detector produces a capture event, and the destination register samples unsync_bus.

The same destination edge registers sync_bus and enable_pulse. SYS_CTRL sees the resulting valid pulse together with the captured byte on a subsequent processing edge. With two synchronizer stages, event latency is several REF_CLK cycles and depends on relative clock phase.

Required stability assumption

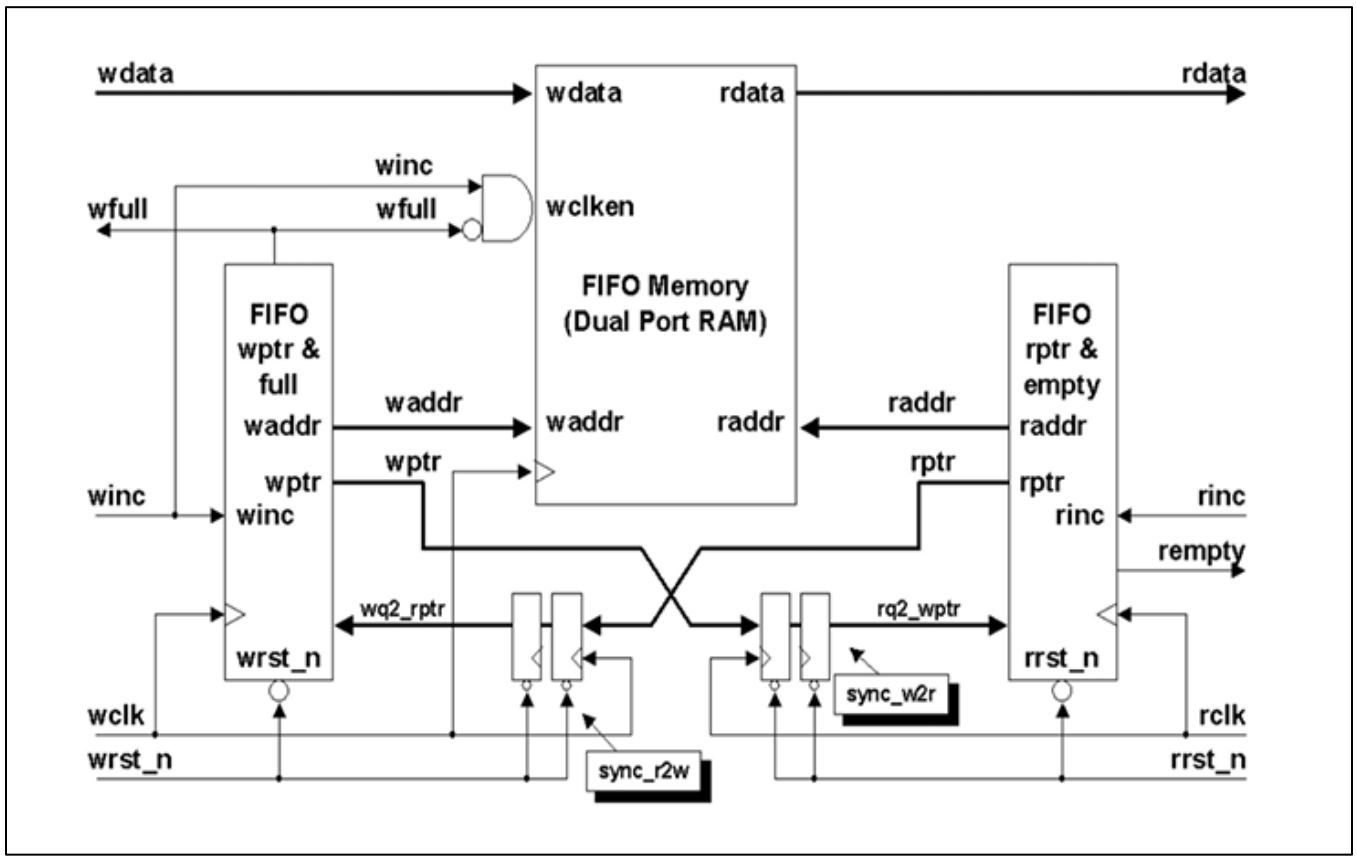
This circuit has no acknowledgement from the destination. The receive byte must remain stable while the enable crosses the synchronizer and until capture. The synchronized enable must return low between bytes. With the default slow UART frame rate and fast REF_CLK, there is a large nominal interval between accepted bytes, but the assumption still needs CDC constraints and verification.

Shared pulse generator

```
pulse = in & ~in_reg
in_reg <= in  // sampled on the local rising clock edge
```

The same Pulse_generator design is used twice: to produce the receive capture event in REF_CLK and to generate the transmit FIFO read event in TX_CLK. It is a combinational edge pulse from a delayed sample, not an independently clocked pulse-stretching block.

1.12 Asynchronous response FIFO



FIFO_top combines storage, read and write pointer blocks, and two Double_FF synchronizers. The memory writes on REF_CLK and reads combinationally from the current read address. The read side runs on TX_CLK. full is interpreted by SYS_CTRL; empty drives UART_TX data_valid through inversion.

A write occurs only when w_inc and not full. A read-pointer increment occurs only when r_inc and not empty. These local guards prevent overwrite and underflow at the pointer interfaces. The FIFO buffers response bytes, not received command bytes.

For depth eight, each memory address uses three bits, while each pointer uses four bits to represent address plus wrap position. The design assumes a power-of-two depth. The full-detection slice also assumes a sufficiently wide pointer and should be reviewed before reducing depth.

FIFO pointers flags and latency

Binary and Gray pointer roles

FIFO_WR and FIFO_RD keep local binary pointers for address generation. Each pointer advances only on an accepted local operation. bin_2_gray converts the binary pointer with Gray = binary XOR (binary >> 1). Adjacent Gray values differ in one bit, making the pointer suitable for transfer through a multi-flop synchronizer when physical bus timing is properly constrained.

The Gray value is registered before crossing to the other domain. In these files, that register samples the Gray encoding of the current binary register, while the binary pointer is updated by another nonblocking assignment on the same edge. The exported Gray pointer therefore trails a binary pointer increment by one local clock.

Empty and full comparisons

```
empty = (local_read_gray == synchronized_write_gray)
full  = upper_two_bits_differ && lower_bits_equal
```

FIFO_RD compares the local current Gray pointer with the synchronized write pointer. FIFO_WR compares the local current Gray pointer against the synchronized read pointer with the upper two bits inverted. This distinguishes the same storage address with opposite wrap status.

Double_FF uses two receiving-clock registers for each pointer bit. Together with the registered source Gray value, this introduces several cycles before a newly written byte becomes visible as nonempty or a newly freed slot becomes visible as not full. Flags can therefore remain conservatively asserted after the other side changes.

Storage behavior

FIFO_Mem clears all entries on write-side reset, accepts a synchronous write, and continuously exposes mem[raddr]. The TX wrapper captures this head byte before the read pointer advances. This is a first-word-visible style interface; there is no separate registered read-data-valid output.

A physical implementation should apply appropriate synchronizer attributes and Gray-bus timing constraints, and verify reset sequencing across both sides. Source code alone does not prove CDC safety or successful memory inference.

1.13 Transmit dequeue and complete flow

How one response byte leaves the FIFO

Step	Event	Consequence
1	FIFO becomes nonempty in TX_CLK domain	~empty raises UART_TX data_valid
2	TX FSM leaves IDLE	P_data_reg has captured the FIFO head; busy rises
3	Pulse_generator detects rising busy	r_inc is asserted for the local edge window
4	Next TX_CLK edge accepts r_inc	FIFO read pointer advances unless empty
5	UART frame continues	Current payload is serialized from P_data_reg
6	TX returns to IDLE	Next queued byte can start a new frame

The FIFO is popped at the beginning of transmission, rather than after the stop bit. The current byte has already been copied into TX storage, so advancing the FIFO head does not alter its serialized payload. Correctness depends on the capture, busy, and read-pulse ordering in this integration.

Example of an ALU request

The master sends CC, 0C, 03, and 00 as four separate UART frames. RX validates each frame and DATA_SYNC transfers each accepted byte. The controller writes 12 and 3 into the operand registers, opens the ALU clock, and issues addition. It captures 0x000F, writes 0x0F into the FIFO, and then writes 0x00.

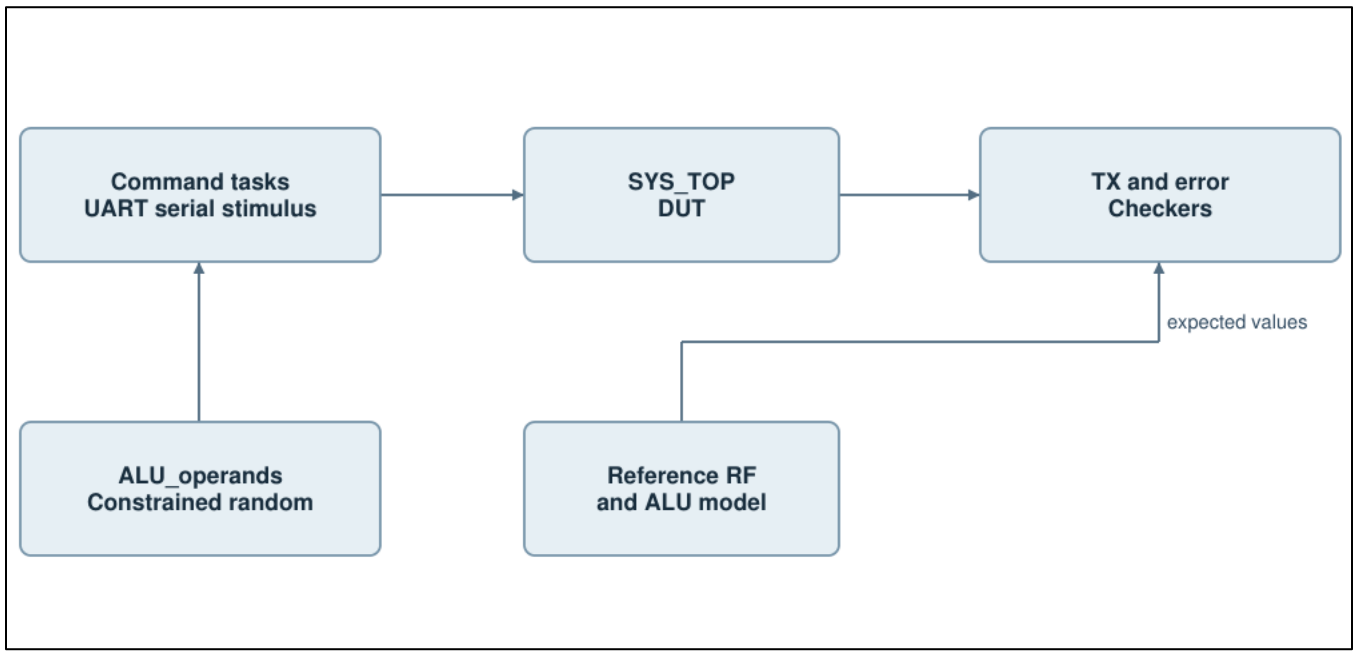
After pointer synchronization, UART_TX emits a frame containing 0x0F and then a frame containing 0x00. Each has its own start, parity if enabled, and stop bit. The master reconstructs the 16-bit result as (second_byte << 8) | first_byte.

Throughput boundary

A register or ALU operation takes only a few fast processing cycles, but UART serialization and FIFO visibility take much longer. FIFO full can delay response packing. The eight-byte FIFO can hold up to eight one-byte responses or four complete two-byte ALU responses, though occupancy is dynamic and no receive flow control is provided.

2. The verification and testing

2.1 Testbench architecture



SYS_TOP_TB.sv declares tb_final_system and imports ALU_operands_pkg from rand_pkg.sv. It creates REF_CLK with a 20 ns period and UART_CLK with an approximately 271.267 ns period. A reference register array stores expected bytes. An ALU reference task computes expected results.

Stimulus tasks build UART frames and higher-level commands. Checkers inspect internal register storage for reset and writes, and reconstruct outgoing serial frames for reads and ALU results. The testbench uses internal DUT hierarchy for TX clock and busy access, so those checkers are coupled to the present instance names.

Randomization constraints

ALU_operands randomizes unsigned eight-bit A_c and B_c and the function enum. It excludes no_op. For division it requires a nonzero divisor and exact divisibility, A_c % B_c == 0. This avoids fractional quotients and division by zero in randomized tests, but also leaves those cases untested.

The initial sequence uses labels in the testcase string for waveform inspection, prints diagnostic messages, and requests a SYS.vcd dump. It ends with $stop. There is no aggregate pass/fail counter or functional coverage model in the supplied testbench.

2.2 Testbench task reference

Task	Function
config_regs	Initialize or update the reference register configuration; does not program DUT registers
print_RF / ref_edit	Print expected state or update one reference register
uart_send_byte	Drive start, eight LSB-first data bits, optional parity, and stop
uart_send_byte_errored	Drive selected parity and stop values for error injection
rf_write_default_config	Send AA, address, and data using default UART timing; update reference RF
rf_read_default_config	Send BB and address using default UART timing
ALU_OP_default_cfg	Send CC with operands or DD without operands
calc_exp_ALU	Compute the expected 16-bit arithmetic or logic result
check_reset / check_writing	Compare the full reference RF against DUT.RF0.reg_file
check_read	Capture one response frame on TX clock falling edges and compare
check_ALU	Capture low and high result frames and compare with packed expected frames
wait_par_error / wait_stop_error	Wait for an error-output event or a completion-flag event

Concurrent stimulus and observation

The register-read scenario forks a command-sending branch and a checking branch. Error injection also forks stimulus and an error watcher. ALU tests send the command and then call the checker, relying on system latency to begin observation before response transmission.

Frame checkers use the internal tx_busy signal to determine frame activity. This is useful for debugging the present design but is less independent than an external serial monitor that detects start bits and samples at the configured bit period. Checker waits do not have a general timeout.

2.3 Implemented verification scenarios

Scenario	Stimulus and check	Scope
Reset	Assert reset; compare default RF contents	Checks storage during reset, not full release behavior
Register writes	Write 0x55 to 0x5, 0x88 to 0x8, 0x0D to 0xD	Compare reference and DUT storage
Register reads	Read 0x5, 0x6, 0x8, and 0xD	Check transmitted frames, including an untouched register
ALU with operands	50 constrained randomized CC commands	Reference ALU result versus two TX frames
ALU without operands	20 randomized function selections using DD	Reuse operands retained from previous operations
Parity error	Send 0xFF with even parity enabled and parity bit one	Observe parity_error event
Stop error	Send 0xFF with parity zero and stop bit zero	Observe stop_error event

All executed command tasks use even parity and BIT_TIME_32. There is no executed programming sequence for configuration addresses 0x2 and 0x3. The specification requests initial configuration operations, but this testbench relies on reset defaults.

What this does not establish

The supplied run contains all 15 supported function names across its 70 ALU checks, as counted on page 29. This establishes observed function occurrence, not exhaustive operand or branch coverage. No formal coverage report was supplied. Odd parity, parity disabled, alternative RX prescales, nondefault divider ratios, FIFO saturation, pointer wrap, reset during a transaction, invalid functions, and invalid-start recovery are not systematically checked by the initial sequence.

Error checks watch flag activity but do not establish that a corrupt frame caused no register write, no command advance, and no response. Those side-effect checks are needed to validate rejection behavior completely.

2.4 Testbench review observations

Parity helper and configuration encoding

uart_send_byte tests even_parity in both branches of its parity calculation. The second branch should correspond to odd_parity; as written, odd parity leaves the automatic local parity value unassigned. This does not affect the default even-parity calls but prevents a reliable odd-parity test.

config_regs assigns the enum directly to ref_RF[2][1:0]. The enum values are no_parity = 0, even_parity = 1, and odd_parity = 2. Hardware expects {PAR_TYP, PAR_EN}, so odd parity enabled requires binary 11, decimal 3. This task changes only the reference model and needs an explicit field mapping before nondefault tests are added.

Frame capture and error watchers

check_ALU always places the second frame at index 11, even when parity is disabled and a frame has only 10 bits. The parity-free case needs a variable frame length. Packed capture arrays are two-state bit arrays, which can hide unknown signal values by conversion.

wait_par_error and wait_stop_error use an any-change event, @(signal), rather than a positive edge or explicit high-value condition. A falling transition can satisfy that wait too. Most failure paths print $display messages rather than $error or $fatal, and there is no summary count. A hung read or ALU wait has no watchdog.

Reference and stimulus limits

calc_exp_ALU has no divide-by-zero guard, while RTL returns 0xFFFF. Randomization avoids this mismatch rather than verifying the policy. ALU_OP_default_cfg updates reference operands even when with_operand is false; current A and B happen to retain the last values, but future changes could make the reference diverge from stored DUT operands.

The write test sends 0x0D at address 0xD although its explanatory printout says 0xDD. Both error sections print the same case number. The source list includes Binary2Gray.v twice. These are documentation or build-hygiene issues, distinct from a demonstrated hardware failure.

2.5 Implementation limits and specification differences

Observation	Practical implication
ALU result is 16 bits in RTL	Use the RTL and SYS_CTRL interface, rather than the specification table that labels the ALU result as eight bits
Normal RF addresses end at 0xF	The specification upper value 0x15 cannot be addressed with this integration
RX uses a generated RX_CLK	Clock-domain diagrams and baud calculations must include CLK_DIV1
RST_SYNC uses correctly named input ports	The specification reset table reverses the verbal clock/reset descriptions
Unsupported ALU function never asserts valid	A command with function 0xF stalls execution until reset
Configuration crosses directly into UART logic	No atomic commit or CDC handshake protects an active-frame update
Receive CDC has no acknowledge	Byte stability and enable separation must be maintained
No command timeout or input queue	Missing or corrupted payloads can leave parsing incomplete
Several widths are hard-coded	Nondefault parameters require additional design review

Additional timing concerns to validate

Changing divider ratios selects clocks through combinational logic and changes active counter thresholds without a reconfiguration protocol. Settings should be updated only through a defined idle procedure, with timing analysis of the clock switch. Source synchronizers release reset on root clocks, not independently on the derived UART clocks.

The receiver counter is held rather than explicitly cleared when disabled; a rejected start can return to IDLE with frame-position history still present. Directed false-start tests should determine the resulting recovery behavior. These are source-review concerns, not claims of measured simulation failure.

2.6 Recommended completion checks

Priority	Check	Evidence to collect
1	Compile unmodified RTL and testbench with a capable SystemVerilog simulator	Build log; correct compilation units and package order
1	Run the existing seven scenarios	Error count, seed, transcript, and waveform
1	Add finite timeouts and fatal failures	Intentional mismatch and missing-response tests fail predictably
1	Verify all ALU functions with directed corners	Zero, 255, carry, underflow, comparisons, complements, and shifts
1	Check division by zero and function 0xF	Documented result or recovery policy with directed evidence
2	Exercise odd and disabled parity	Correct stimulus encoding and 10 versus 11 bit capture
2	Exercise RX prescales 8, 16, and 32	Maintain 115200 bit/s with current divider mapping
2	Fill and drain FIFO across wraparound	No lost, duplicated, or reordered response bytes
2	Inject start glitches and corrupted payloads	Correct recovery and absence of unintended side effects
2	Reset during RX, execution, and TX	Deterministic reset assertion and release behavior
3	Review CDC and generated-clock constraints	Synchronizer recognition, Gray bus timing, and clock relationships

The supplied results provide an initial successful run of the existing scenarios; rerunning them remains useful after changes. The other checks extend that evidence. They were not added to the supplied sources during this task. The implementation is preserved, and the waveform evidence is reproduced in the following results chapter.

2.7 Source inventory

Source file	Declared module	Responsibility
system_top.v	SYS_TOP	System integration
SYS_CTRL.v	SYS_CTRL	Command parser and response control
Register_File.v	Register_File	Operands, UART configuration, storage
ALU.v	ALU	Registered arithmetic and logic
CLK_gate.v	CLK_gate	Latch-based ALU clock enable
Integer_CLK_DIV.v	Integer_CLK_DIV	TX and RX divided clocks
RST_SYNC.v	RST_SYNC	Reset assertion and release pipeline
UART_TOP.v	UART_TOP	TX and RX wrapper
UART_TX.v	UART_TX	Transmit datapath integration
FSM_CU_TX.v	FSM_TX	Transmit state machine
parallel_2_serial_TX.v	parallel_2_serial_TX	LSB-first serialization
Parity_gen_TX.v	Parity_gen_TX	Transmit parity generation
MUX_TX.v	MUX_TX	Frame-bit source selection
UART_RX.v	UART_RX	Receive datapath integration
FSM_CU_RX.v	FSM_CU_RX	Receive state machine
edge_bit_counter_RX.v	edge_bit_counter_RX	Oversampling and frame position
sampler_RX.v	sampler_RX	Three-sample majority decision

Module names are case-sensitive. A source filename is not always the name to select as the simulation top: choose tb_final_system for the supplied testbench and SYS_TOP for the synthesizable system.

2.8 Source inventory and reproducibility

Source file	Declared module or package	Responsibility
Serial_2_parallel_RX.v	Serial_2_parallel_RX	Payload reconstruction
Start_check_RX.v	Start_check_RX	False-start check
Parity_check_RX.v	Parity_check_RX	Receive parity accumulator
Stop_check_RX.v	Stop_check_RX	Stop-bit validation
DATA_SYNC.v	DATA_SYNC	Receive bus transfer
Multi_FF.v	Multi_FF	Control-level synchronizer
Pulse_generator.v	Pulse_generator	Local rising-edge pulse
FIFO_top.v	FIFO_top	Response FIFO integration
FIFO_Mem.v	FIFO_Mem	FIFO storage
FIFO_WR.v	FIFO_WR	Write pointer and full flag
FIFO_RD.v	FIFO_RD	Read pointer and empty flag
Binary2Gray.v	bin_2_gray	Pointer encoding
sync.v	Double_FF	Two-stage pointer transfer
rand_pkg.sv	ALU_operands_pkg	Random operand and function class
SYS_TOP_TB.sv	tb_final_system	System stimulus and checks
src_files.list	Source list	Build input ordering

Build the package before the testbench and compile the .v sources as Verilog where needed. Several Verilog ports use names such as bit and type that are reserved in SystemVerilog; forcing every source into SystemVerilog mode can therefore cause parser errors. Remove the duplicate Binary2Gray.v list entry if the chosen tool treats duplicate declarations as errors.

Source references: Final_System(1).pdf, pages 1-15; UART_RX.pdf, pages 1-6; UART_TX_Spec_Document.pdf, pages 1-5; block_diagram.PNG and Block_Diagram(1).PNG. Source modules listed on pages 26-27 are the primary basis for implemented behavior. Saeed_Maher_System.pdf adds the supplied test descriptions, waveforms, and recorded checker outcomes. No timing, area, power, or formal coverage report was provided.

2.9 Reported simulation results

Evidence supplied by Saeed Maher Saeed in Saeed_Maher_System.pdf

The supplied results report describes seven test scenarios, presents waveform screenshots on source pages 6-10, and records checker output on source pages 11-20. This chapter summarizes the reported outcomes and includes the waveform figures. The full transcript is omitted to keep the documentation manageable.

Test	Reported outcome	Evidence
1 Reset	Successful RF reset comparison	Default REG2 = 0x81 and REG3 = 0x20; other entries zero
2 RF writes	Successful full RF comparison	0x5 = 0x55, 0x8 = 0x88, 0xD = 0x0D
3 RF reads	4 expected and actual frames match	Reads at 0x5, 0x6, 0x8, and 0xD
4 ALU with operands	50 successful result checks	CC commands with randomized A, B, and function
5 ALU stored operands	20 successful result checks	DD commands using A = 250 and B = 5
6 Parity error	Error assertion reported	0xFF with incorrect even parity bit one
7 Stop error	Error assertion reported	0xFF with correct parity zero and stop zero

The recorded output contains 74 expected-versus-actual serial comparisons: four register-read frames and 70 ALU response pairs. All 74 printed comparisons match. The run also reports successful reset and register-write comparisons, parity-error detection, stop-error detection, and completion of the test sequence.

Evidence boundary

These are results recorded in the supplied PDF, not a fresh simulation performed for this report. The PDF does not provide a reproducible seed, tool-version record, aggregate simulator error summary, raw waveform database, or formal coverage database. Existing checker limitations on page 23 still apply. A successful printed check does not establish behavior outside the exercised conditions.

The source test description lists no operation among tested functions, but rand_pkg.sv excludes no_op and the recorded operation names contain no such transaction. The results here therefore count only the 15 supported operations actually observed.

2.9.1 Observed ALU operation counts

Function	With operands	Stored operands	Total
Add	5	1	6
Subtract	6	0	6
Multiply	2	3	5
Divide	2	3	5
AND	2	2	4
OR	1	2	3
NAND	5	0	5
NOR	1	2	3
XOR	7	0	7
XNOR	5	0	5
Equal	0	3	3
Greater than	2	1	3
Less than	7	0	7
Shift right	4	2	6
Shift left	1	1	2
Total	50	20	70

Counts are derived from the operation names in the supplied recorded output, rather than from a functional coverage database. All 15 supported functions appear at least once across both groups. The with-operands group does not contain equality comparisons; equality appears three times in the stored-operands group. Several stored-operand functions are absent, so neither group independently covers every operation.

Representative recorded results include 227 + 112 = 339 (0x0153), 238 x 180 = 42840 (0xA758), 250 / 5 = 50 (0x0032), and 250 shifted left = 500 (0x01F4). The last two use the stored operands. Each result is compared as a low-byte frame followed by a high-byte frame.

2.9.2 Reset and register write waveforms

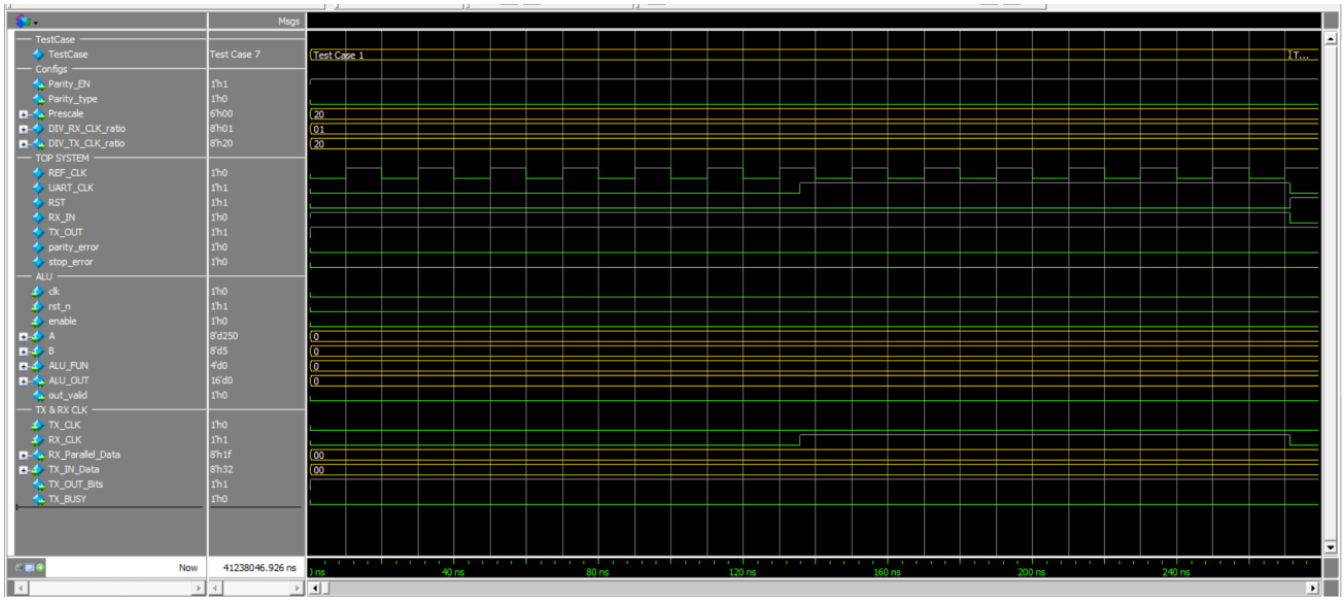


Figure R1. Reset scenario. Source results page 6. The capture shows system clocks, reset, configuration, ALU controls, and UART signals around Test Case 1.

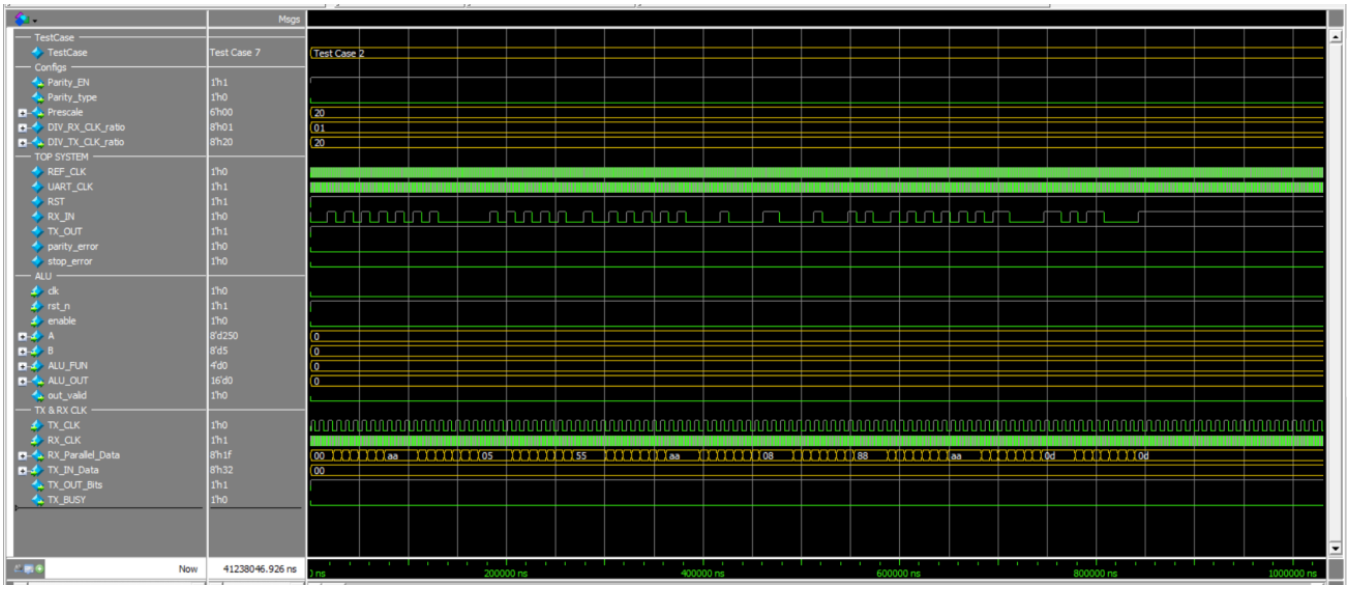


Figure R2. Register-write command activity. Source results page 6. Serial input activity accompanies the write scenario while the system clocks continue running.

The recorded checker confirms that reset initializes the full register file and that the three UART write commands produce the expected storage values. These screenshots provide timing context; the storage view on the next page makes the written values easier to inspect.

2.9.3 Register storage and readback evidence

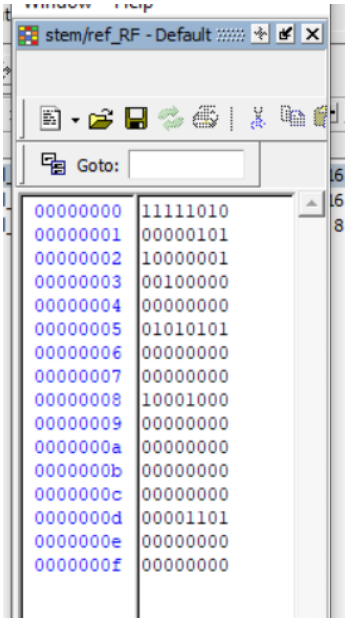


Figure R3. Register memory view. Source results page 7. Highlighted locations are address 0x5 = 0x55, 0x8 = 0x88, and 0xD = 0x0D.

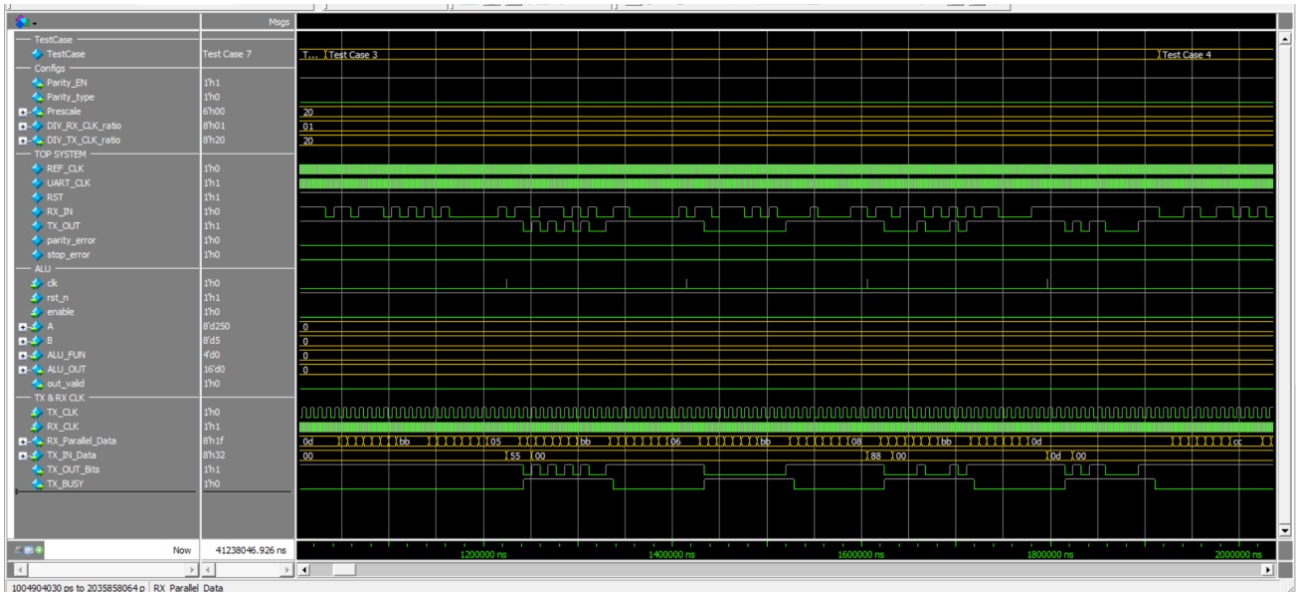


Figure R4. Register-read waveform. Source results page 7. Test Case 3 drives read commands and observes outgoing serial response activity.

The four read checks report matching frames for addresses 0x5, 0x6, 0x8, and 0xD. Their payload bytes are 0x55, 0x00, 0x88, and 0x0D respectively. Reading untouched address 0x6 demonstrates the zero-initialized value in this run.

2.9.4 ALU command and response waveforms

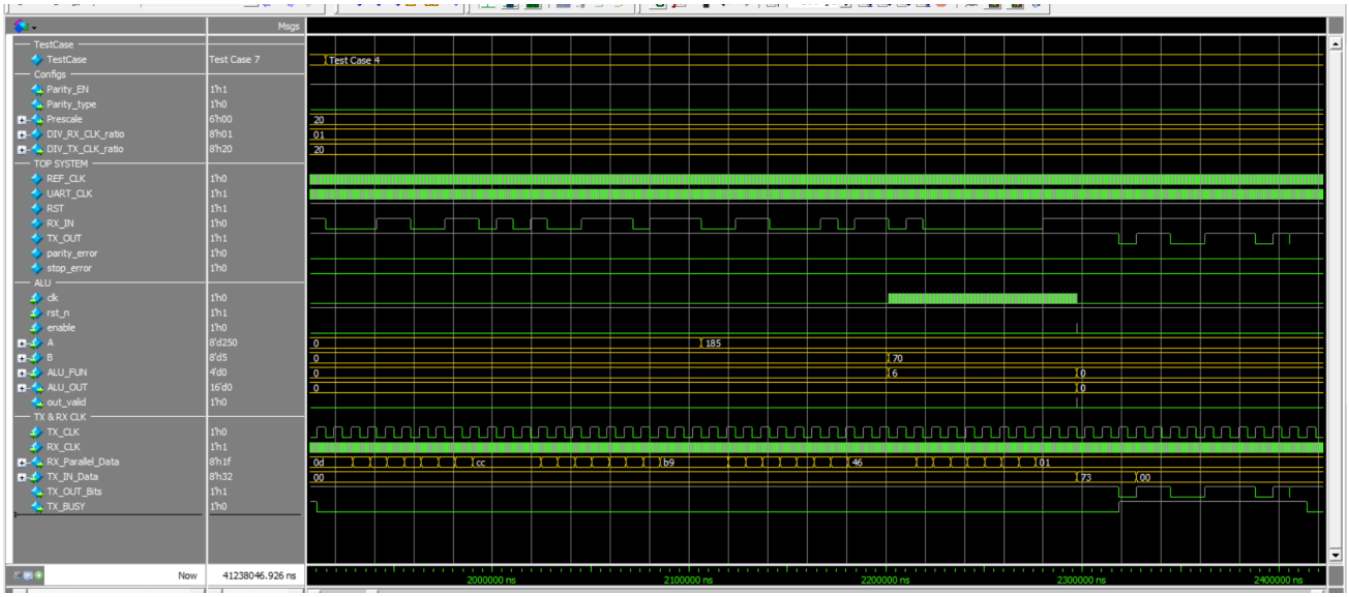


Figure R5. ALU with operands. Source results page 8. The wider view shows the command stimulus and subsequent system activity in Test Case 4.

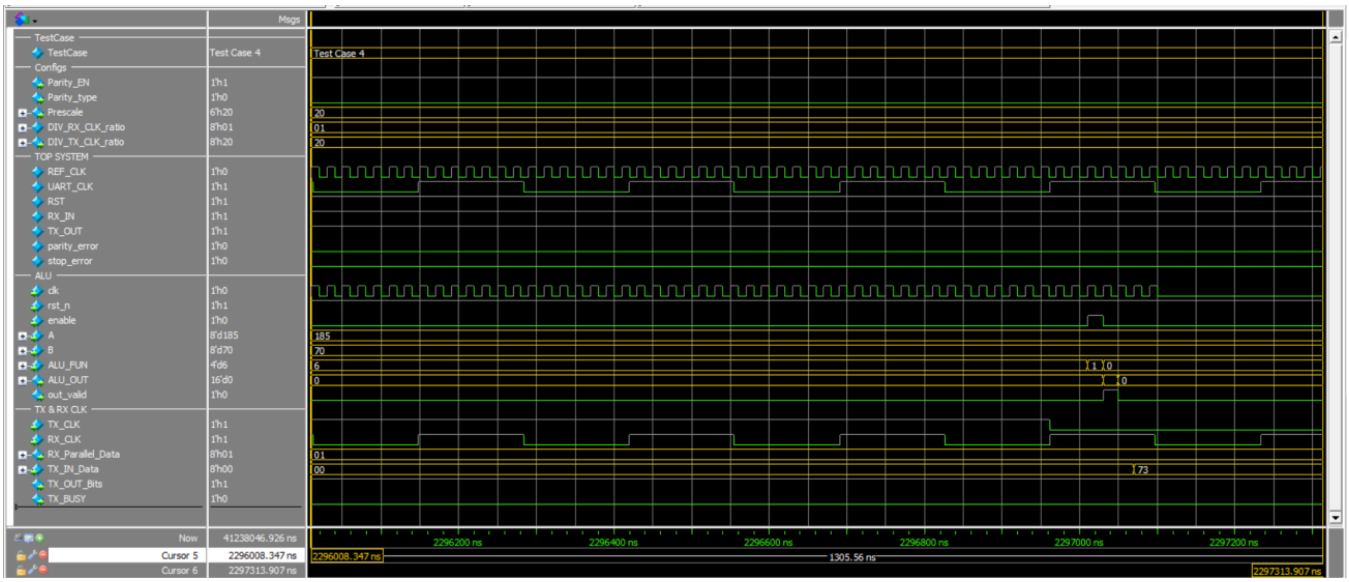


Figure R6. ALU execution and response overview. Source results page 8. The annotated regions highlight execution-related activity and the later transmit sequence.

These captures accompany the 50 reported successful with-operands checks. The receive and transmit interfaces operate at the UART rate, while ALU execution is much shorter in the processing domain. The next page includes the closer execution view.

2.9.5 ALU execution and stored operand evidence

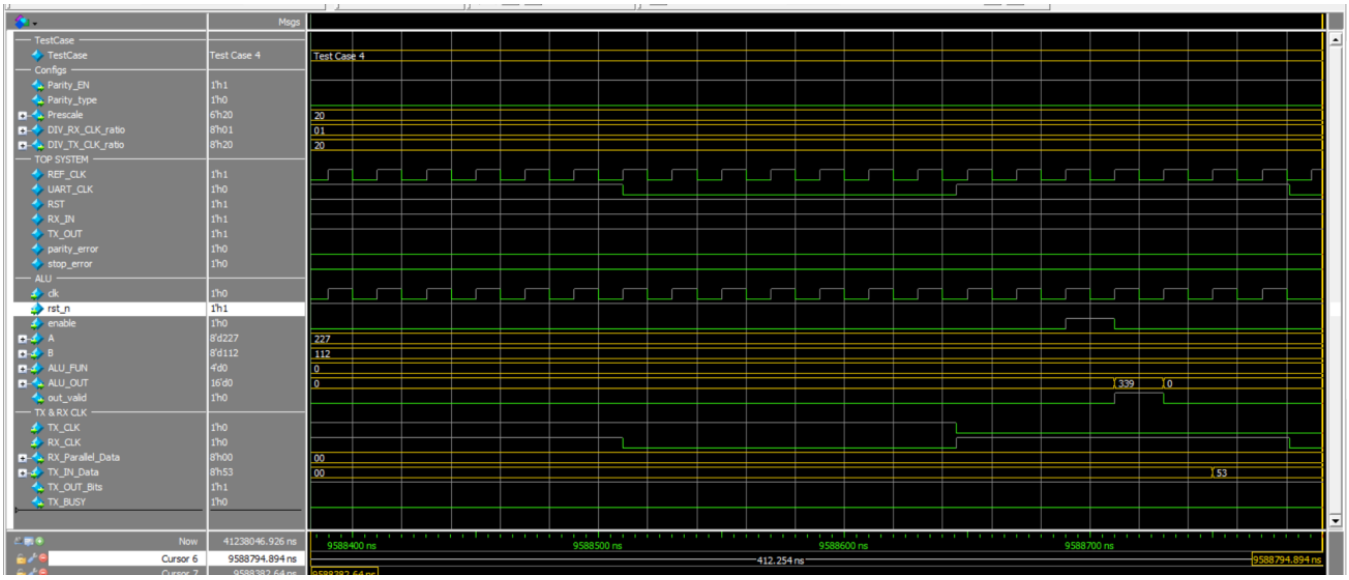


Figure R7. Detailed addition example. Source results page 8. The trace labels show A = 227, B = 112, function zero, a valid result of 339, and later low-byte transmit input 0x53.

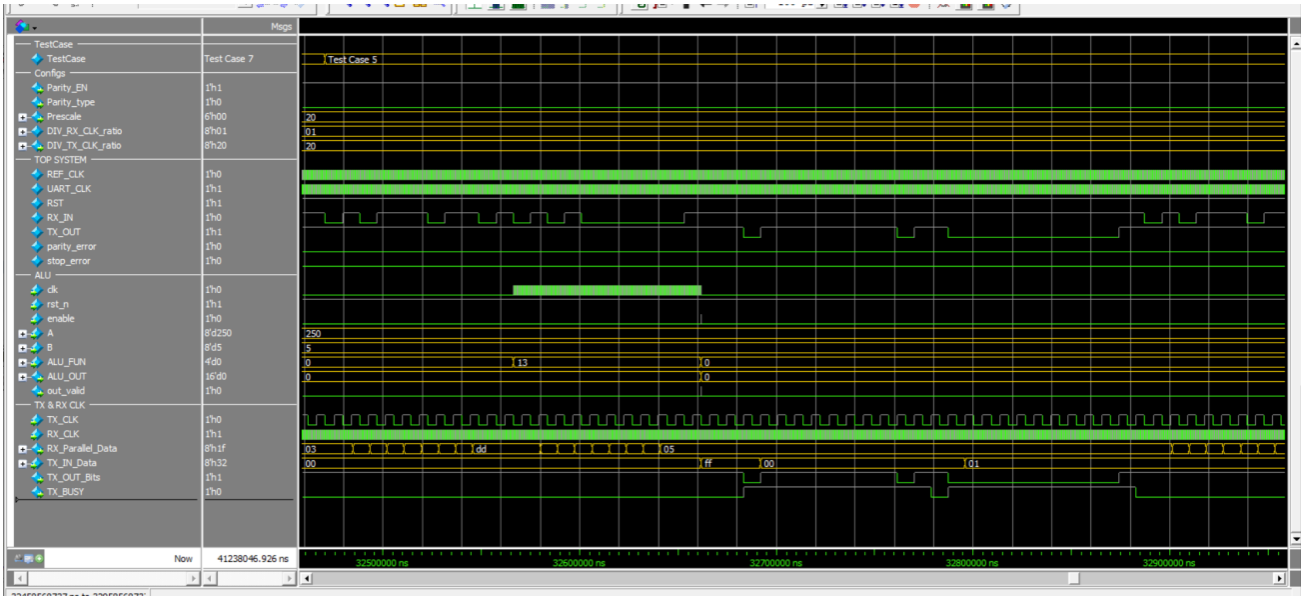


Figure R8. ALU operation without new operands. Source results page 9. Test Case 5 reuses stored operands while selecting new functions.

For the addition example, 339 = 0x0153, so the response payload order is 0x53 then 0x01. In the stored-operand group, the recorded operands remain 250 and 5 for 20 function selections. This is evidence for operand reuse in that run, not randomized stored-register histories.

2.9.6 UART error detection waveforms

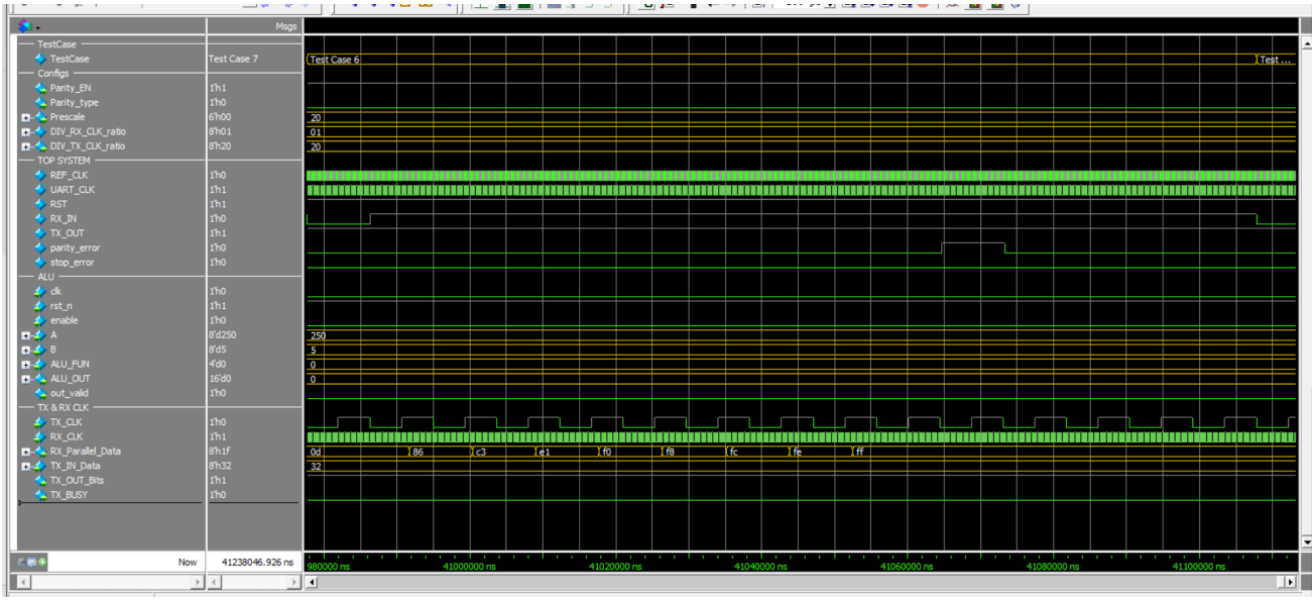


Figure R9. Parity-error injection. Source results page 9. The annotated receiver error activity accompanies a 0xFF frame with an incorrect even-parity bit of one.

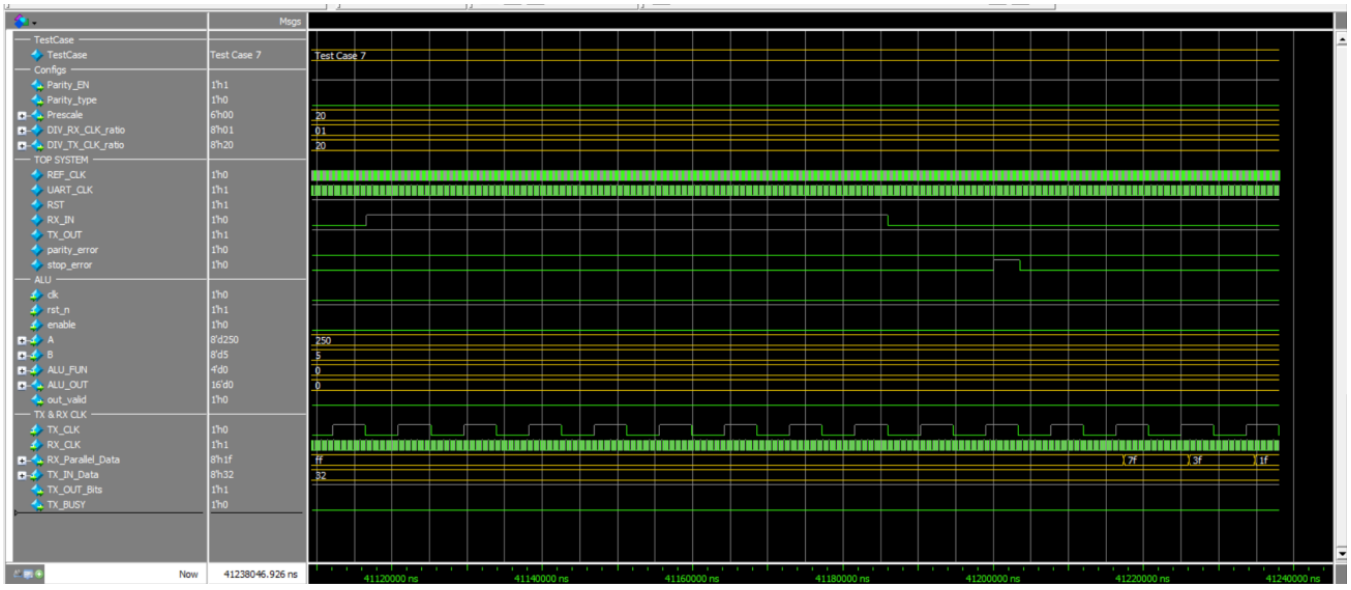


Figure R10. Stop-error injection. Source results page 10. The annotated stop_error activity accompanies a frame whose stop bit is zero.

3. Synthesis & formality post synthesis

3.1 Synthesis flow and constraints

The synthesis run uses Synopsys Design Compiler O-2018.06-SP1 and top design SYS_TOP. The script analyzes the Verilog modules, elaborates and links the hierarchy, runs check_design, creates input and output path groups, sources cons.tcl, and invokes compile. SYS_TOP.svf records synthesis guidance for the later Formality run.

Setup item	Recorded setting
Standard cell libraries	scmetro_tsmc_cl013g_rvt at SS, TT, and FF corners; all three are in target_library and link_library
Maximum delay corner	SS, 1.08 V, 125 C
Minimum delay corner	FF, 1.32 V, -40 C
Reference root clock	20 ns period; 50 MHz
UART root clock	217.267 ns period; approximately 4.603 MHz
Generated clocks	ALU divide by 1; TX divide by 32; RX divide by 1
Uncertainty and transition	Setup 0.2 ns; hold 0.1 ns; transition 0.05 ns; clock latency zero
Clock relationship	UART root and generated clocks asynchronous to REF and gated ALU clocks
External interface	RX input delay = 20% of UART period; TX and error output delays = 30%
Drive and load	RX driven by BUFX2M at SS; load value 0.1 on all outputs in library units

Clock constraint difference

The UART period is 217.267 ns in both cons.tcl and the echoed synthesis log. The system specification and testbench use approximately 271.267 ns, corresponding to 3.6864 MHz. The synthesis constraint therefore targets a faster UART source. Its divide-by-32 TX clock is approximately 143.8 kHz, rather than the nominal 115.2 kHz. This is a constraint difference, not evidence that the applied hardware clock changed.

Generated-clock targets are selected with hierarchical get_ports expressions in this DC script. No corresponding selection error is shown in syn.log. Validate the target object type and clock coverage when exporting constraints to another tool. The constraints describe default divider settings, not every programmable configuration.

3.2 Synthesis results

The supplied log reaches Optimization Complete and successfully records hierarchical Verilog, SDF, and SDC writes. No fatal synthesis error is shown. The following numerical values come from the final compile progress table or explicit Tcl return values, rather than from standalone QoR reports.

Metric	Recorded result	Interpretation
Final compile area	21700.7	Library area units; physical units are not established by the supplied log
Worst negative slack column	0.00	No negative setup slack displayed at final optimization; exact positive margin is unavailable
Total setup cost	0.0	Final compile summary value
Design rule cost	0.0	Final compile summary value; does not replace constraints.rpt
Edge triggered register count	356	Return value from all_registers -edge_triggered
Scan chain estimate	4	356 / 100 + 1 using integer arithmetic; estimate only

Exported outputs

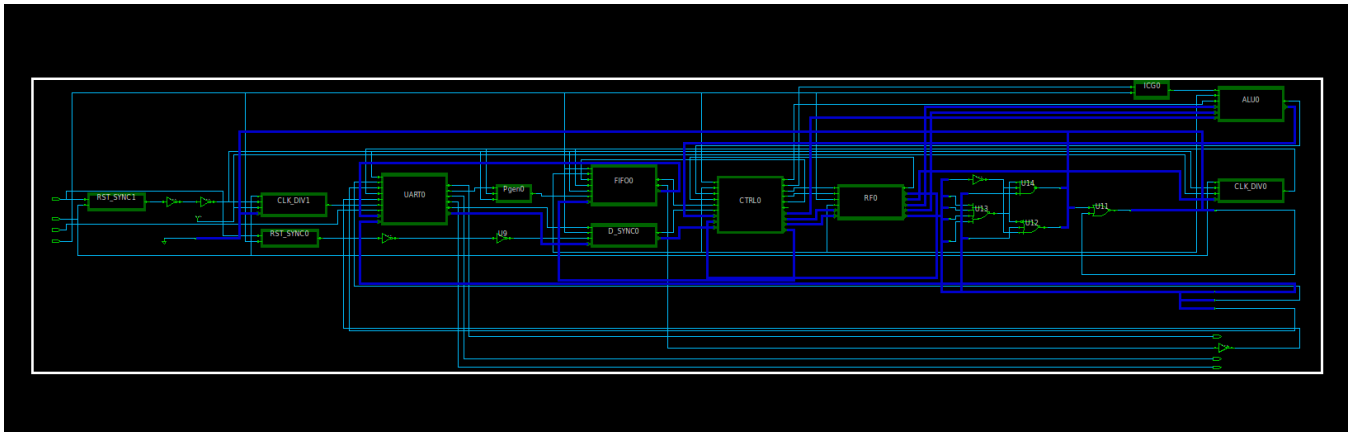
System_Netlist_pre_DFT.v is the implementation netlist read by Formality. System.sdf provides exported delay information; System.sdc provides exported constraints. The script also requests hierarchy area and power reports, the 20 maximum-delay and minimum-delay paths, clock attributes, and all constraint violators. Those separate report files were not supplied, so cell count, power, detailed setup and hold margins, and unconstrained-path coverage cannot be concluded here.

Script details requiring correction or follow up

System_Netlist_pre_DFT.ddc is written with -format verilog. It therefore contains Verilog text despite its .ddc suffix; the log explicitly confirms a Verilog write. A native Design Compiler database requires the DDC output format. This does not affect the .v netlist used in the supplied equivalence run.

The DFT preparation code only counts registers and estimates a number of chains. It does not insert scan, replace flip-flops with scan cells, or demonstrate a DFT rule check. The supplied implementation remains a pre-DFT synthesis result.

Schematic:



Area report:

```
 1
 2 ****************************************
 3 Report : area
 4 Design : SYS_TOP
 5 Version: O-2018.06-SP1
 6 Date   : Mon Oct 19 01:50:08 2026
 7 ****************************************
 8
 9 Library(s) Used:
10
11     scmetro_tsmc_cl013g_rvt_ss_1p08v_125c (File: /home/ICer/project/std/scmetro_tsmc_cl013g_rvt_ss_1p08v_125c.db)
12
13 Number of ports:                          732
14 Number of nets:                         2654
15 Number of cells:                        1907
16 Number of combinational cells:          1498
17 Number of sequential cells:              370
18 Number of macros/black boxes:              0
19 Number of buf/inv:                       246
20 Number of references:                     18
21
22 Combinational area:             12441.249299
23 Buf/Inv area:                     922.532821
24 Noncombinational area:           9259.452187
25 Macro/Black Box area:               0.000000
26 Net Interconnect area:      undefined  (No wire load specified)
27
28 Total cell area:                21700.701486
29 Total area:                 undefined
30 1
```

Worst Setup Slack:

```
93   data required time                                    19.64
94   data arrival time                                    -15.42
95   ---------------------------------------------------------------------------
96   slack (MET)                                           4.22
97
```

Worst Hold Slack:

```
43   ---------------------------------------------------------------------------
44   data required time                                    0.08
45   data arrival time                                    -0.91
46   ---------------------------------------------------------------------------
47   slack (MET)                                           0.83
48
```

3.3 Post synthesis Formality verification

Formality L-2016.03-SP1; report timestamp 2 October 2026

fm_script.tcl enables synopsys_auto_setup and reads SYS_TOP.svf. It builds the reference container from RTL and the implementation container from System_Netlist_pre_DFT.v, loads the three standard-cell databases into both containers, selects SYS_TOP, and executes match followed by verify.

Verification item	Result
Overall equivalence status	Verification SUCCEEDED
Reference and implementation	REF:/WORK/SYS_TOP and IMP:/WORK/SYS_TOP
Matched compare points	360, all matched by name
Matched input or black box output objects	4; separate from the 360 compare points
Unmatched compare points and input objects	0 in both reference and implementation
Passing points by type	356 DFF, 1 latch, and 3 ports; total 360
Failing compare points	0
Aborted compare points	0
Unverified compare points	0
Not compared points	0

Evidence and meaning

The summary in fm.log and the 360 entries in passing_points.rpt agree. failing_points.rpt, aborted_points.rpt, unverified_points.rpt, and umcompared_points.rpt each report no points in their respective categories. The filename umcompared_points.rpt contains a spelling error, but its report type is not_compared_points.

This establishes RTL-to-netlist logical equivalence for the models, library interpretation, guidance, and setup assumptions used in the supplied run. It does not establish timing closure, CDC safety, or that the RTL implements every intended requirement. Simulation and equivalence answer different questions and remain complementary.

```
********************************** Verification Results **********************************
Verification SUCCEEDED
   ATTENTION: synopsys_auto_setup mode was enabled.
              See Synopsys Auto Setup Summary for details.
------------------------------------------------------------
 Reference design: REF:/WORK/SYS_TOP
 Implementation design: IMP:/WORK/SYS_TOP
 361 Passing compare points
------------------------------------------------------------------------------------------
Matched Compare Points     BBPin    Loop   BBNet     Cut    Port     DFF     LAT   TOTAL
------------------------------------------------------------------------------------------
Passing (equivalent)           0       0       0       0       3     357       1     361
Failing (not equivalent)       0       0       0       0       0       0       0       0
******************************************************************************************
```

4. DFT & formality post DFT

4.1 DFT constraints and test setup

The DFT run analyzes the revised rtl_dft source tree and uses the same SYS_TOP design and three standard-cell corners as synthesis. It invokes compile -scan, defines a test protocol, runs pre-insertion DFT checks, previews and inserts scan chains, and performs incremental scan optimization. Post-insertion dft_drc requests a coverage estimate. This is a separate run from the pre-DFT synthesis chapter.

Constraint or signal	Intended setting
Scan architecture	Full scan; multiplexed flip-flops; replacement enabled; no clock mixing; maximum chain length 100
Scan clock and reset	SCAN_CLK test waveform at times 30 and 60; SCAN_RST active low
Test mode and shift enable	test_mode active high and held at 1 in test setup; SE active high for scan shifting
Scan data interface	SI input vector and SO output vector; four lanes shown by scan preview
Test protocol timing	Period 100 ns; input and bidirectional delays zero; strobe 20 ns; strobe width zero
Intended STA scan clock	100 ns period, equivalent to 10 MHz
STA interface delays	SI, SE, test_mode inputs and SO outputs: 20 ns relative to intended scan clock
Functional clocks and UART delays	REF 20 ns; UART 217.267 ns; UART input and output delays 20% of UART period
Shared electrical setup	Setup uncertainty 0.2 ns; hold 0.1 ns; transition 0.05 ns; latency zero; output load 0.1 in library units
Timing mode selection	set_case_analysis 1 on test_mode; scan group intended asynchronous to REF and UART groups

Intended timing versus applied timing

The STA command selects Scan_CLK, while the DFT protocol correctly selects SCAN_CLK. Port names are case-sensitive. The log reports that Scan_CLK is missing, so the STA scan-clock creation fails. Scan-related I/O delays and the three-group asynchronous command also report empty-object errors. Mixed clock-attribute commands warn that SCAN_CLK is missing. The intended constraints above were therefore not all applied successfully.

The DFT protocol nevertheless identifies SCAN_CLK at 30.0 and 60.0 and accepts SCAN_RST. Test-protocol creation and STA clock creation are distinct operations: successful scan insertion does not repair the failed timing setup. The earlier UART-period mismatch of 217.267 ns versus the testbench value also remains.

4.2 Scan insertion and DFT violations

Result	Reported value
Scan methodology	full_scan with multiplexed_flip_flop style
Scan chain preview	4 chains; SI[3:0] to SO[3:0]
Preview chain lengths	89 cells in each chain; total 356 scan cells
Clock for all preview chains	SCAN_CLK, rising edge at time 30.0
Post insertion status	Design has scan chains and is scan routed
Sequential cell classification	356 valid scan cells plus 1 cell with a violation; 357 total
Pre insertion DFT DRC	1 violation, TEST-505
Post insertion DFT DRC	1 violation, TEST-505; TEST-124 summary warning

Remaining violation

Both DRC runs identify ICG0/U0_TLATNCAX12Mv as having a constant value of one. The DRC classifies this as an other violation, not a broken scan-chain report. The 356 flip-flops are classified as valid scan cells, while the clock-gating latch is the single sequential cell with a violation.

A clock-gating latch forced high is consistent with keeping the gate open during scan so that clock pulses reach downstream registers. The revised clock-gate RTL and mapped netlist were not supplied, so the exact mechanism must be confirmed there. The report should retain this violation and its test-mode context; it is not a clean zero-violation result or an automatically justified waiver.

Constraint and optimization issues

The missing Scan_CLK STA object causes six CMD-036 errors: clock creation, clock grouping, three scan input-delay commands, and the scan output-delay command. These setup errors are separate from the single TEST-505 DFT-rule violation. Incremental compile also reports 70 potential design problems through LINT-99, and repeats the library-corner OPT-998 warning.

Correct the port spelling to SCAN_CLK, rerun the constraint stage, and inspect clock and unconstrained-path reports before claiming scan timing closure. The supplied log shows scan insertion and routing completed despite the constraint errors. A preview gives the 4 x 89 architecture; an exported final scan-path report would provide independent confirmation after incremental optimization.

4.3 DFT coverage estimate and implementation results

The post-insertion command dft_drc -verbose -coverage_estimate produces an uncollapsed stuck-fault summary with 16142 total faults and 99.22% reported test coverage. This is a DFT-tool estimate under the supplied test protocol, not a delivered production ATPG pattern set or a functional coverage metric.

Fault class	Code	Count
Detected	DT	15789
Possibly detected	PT	1
Undetectable	UD	229
ATPG untestable	AU	111
Not detected	ND	12
Total faults		16142
Reported test coverage		99.22%

The detected count divided by all listed faults is approximately 97.81%. The reported 99.22% is consistent with a test-coverage denominator that excludes the 229 undetectable faults: 15789 / (16142 - 229) is approximately 99.22%. The summary alone does not specify the treatment of the single possibly detected fault. Do not relabel the reported test coverage as 99.22% detection of all 16142 faults.

The Pattern Summary Report lists zero internal patterns. No test-pattern files, fault-detail report, or customized-protocol ATPG run was supplied. The tool notes that a customized protocol and test simulation library may produce a different coverage result. The AU, ND, and PT classes remain visible in this summary rather than being omitted.

Final incremental compile display	Value
Area	24506.0 in library area units
Worst negative slack and setup cost	0.00 and 0.0
Design rule cost	0.5 remaining; detailed violator list unavailable

The nonzero design-rule cost is a separate optimization metric from TEST-505. Failed scan timing constraints prevent treating the zero displayed setup cost as scan timing signoff. Outputs include System_Netlist_with_DFT.v, SDF, SDC, and requested QoR reports. The .ddc-suffixed output is again written as Verilog, not a native DDC database.

4.4 Post DFT Formality result and proof scope

Formality L-2016.03-SP1; report timestamp 2 October 2026 at 04:19:06

The script reads revised rtl_dft sources as the reference, System_Netlist_with_DFT.v as the implementation, and DFT/SYS_TOP.svf as guidance. Formality reports Verification SUCCEEDED. That result applies to the effective constrained setup below, which differs from the manual script comments.

Verification category	Reported result
Passing compare points	337: 334 DFF and 3 output ports
Failing, aborted, and unverified	0 in each category
Unmatched compare points	0 in both reference and implementation
Not compared	27: 22 unread DFF, 4 excluded SO ports, 1 constant latch
Undriven-net warning	4 reference nets and 12 implementation nets
Effective SVF constants	SE = 0 and test_mode = 1 in both containers

Why the intended constants did not take effect

The manual commands use Ref:/WORK/* and Imp:/WORK/* names. All six commands for excluding SO and setting test_mode and SE return FM-036 Unknown name. The loaded containers are REF and IMP. Use exact resolved object names rather than assuming the attempted mixed-case names selected the intended objects.

During match, SVF processing explicitly sets SE to zero and test_mode to one on both models, and excludes the four reference SO ports. Thus the recorded proof holds with scan shifting disabled but test mode asserted. It does not demonstrate the intended normal-mode case test_mode = 0. The test_mode = 1 setup is also consistent with the DFT constraint file and test-signal declaration, which both select one.

Interpretation of success

The 337 compared points are equivalent under these assumptions. The 27 not-compared points are not failures, but they are outside the explicit compare result. The result does not verify scan shifting because SE is zero, and it does not establish unrestricted functional-mode equivalence. Correct the object selections, establish the required mode after guidance processing, report effective constants and exclusions, and rerun the intended normal-mode proof.

The reference is the revised DFT RTL, not the original no-DFT RTL from the earlier simulation chapter. Keep both source snapshots and the exact SVF/netlist together when interpreting or reproducing the two equivalence runs.

4.5 Why Formality reports unread registers

Unread means the verification model has no relevant active downstream reader for a state element under the current constants and comparison scope. Its fanout can terminate at unused nets, a branch disabled by a constant, or an excluded output. This differs from failing, unverified, and unmatched. The post-DFT result identifies 22 unread registers, and the endpoint report explains where their fanout stops.

Not compared group	Count	Connection to the active setup
CLK_DIV0 registers	9	One divided-clock register plus eight counter bits
CLK_DIV1 registers	9	One divided-clock register plus eight counter bits
RST_SYNC0 and RST_SYNC1	4	Two reset pipeline registers per synchronizer
SO outputs	4	Explicitly excluded by accepted SVF guidance
Clock-gating latch	1	Classified as a constant register
Total not compared	27	22 unread DFF plus 4 excluded ports plus 1 constant latch

What the unread endpoint report actually shows

unread_endpoints(1).rpt lists 71 blocking objects in the fanout. These are diagnostic nets and blocking locations, not 71 unread registers or 71 failed points. Counting its entries gives 61 blocked-by-constant objects and 10 no-reader objects. Examples include reference SE blocked by a constant, SI[3:0] with no reader in the reference model, unused divider carry nets, and constant-blocked implementation nets.

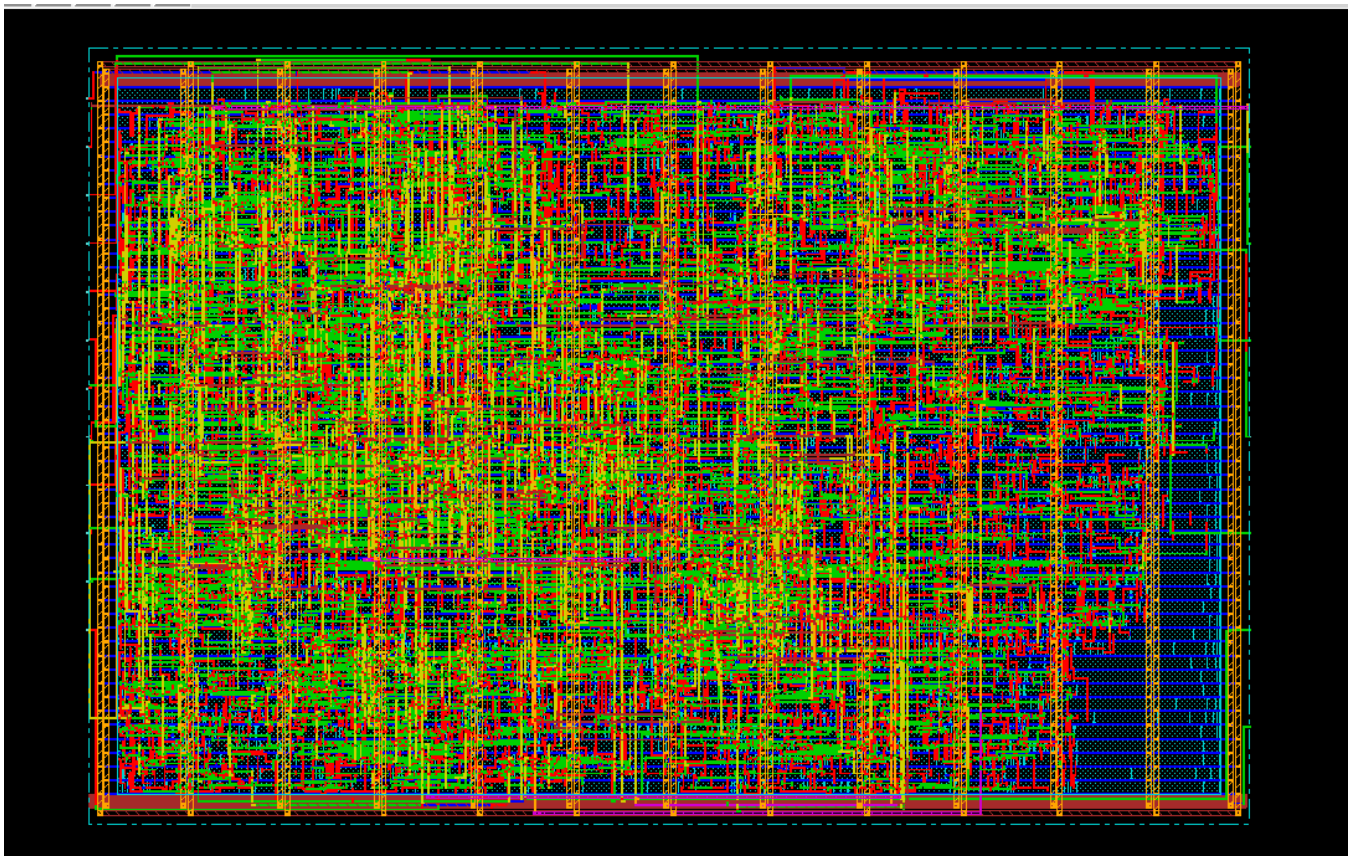
SE = 0 disables the scan-input path, and excluded SO outputs remove scan-only observation points. With test_mode = 1, test clock and reset selection can bypass the functional dividers and reset synchronizers; that is consistent with all 18 divider registers and four reset registers becoming unread. The revised RTL/netlist is required to confirm the exact mux paths. The constant clock-gating latch is accounted for separately from those 22 unread DFF.

Required conclusion and follow up

The unread report is meaningful here; it is not the earlier post-synthesis message No unread cpoints specified. The supplied post-DFT run succeeded for a restricted test-mode setup. Confirm that each bypass and exclusion is intended, then rerun with resolved constants for normal mode. Separately validate scan-chain operation and complete ATPG/test simulation; the present Formality result does not replace those tests.

5. The Physical implementation

5.1 The connections:



5.2The timing results:

Setup:

```
------------------------------------------------------------
          timeDesign Summary
------------------------------------------------------------

+--------------------+---------+---------+---------+---------+---------+--------
-+
|     Setup mode     |   all   | reg2reg | in2reg  | reg2out | in2out  | clkgate
 |
+--------------------+---------+---------+---------+---------+---------+--------
-+
|           WNS (ns):|  5.576  |  5.576  | 18.249  | 77.566  |   N/A   | 17.446
 |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |  0.000  |   N/A   |  0.000
 |
|    Violating Paths:|    0    |    0    |    0    |    0    |   N/A   |    0
 |
|          All Paths:|  1074   |  1059   |   364   |    7    |   N/A   |    1
 |
+--------------------+---------+---------+---------+---------+---------+--------
```

Hold:

```
------------------------------------------------------------
          timeDesign Summary
------------------------------------------------------------

+--------------------+---------+---------+---------+---------+---------+---------+
|     Hold mode      |   all   | reg2reg | in2reg  | reg2out | in2out  | clkgate |
+--------------------+---------+---------+---------+---------+---------+---------+
|           WNS (ns):|  0.050  |  0.058  |  0.050  | 20.773  |   N/A   |  0.555  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |  0.000  |   N/A   |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |    0    |   N/A   |    0    |
|          All Paths:|  1074   |  1059   |   364   |    7    |   N/A   |    1    |
+--------------------+---------+---------+---------+---------+---------+---------+
```

5.3 The connectivity and geometry verification

```
  VERIFY GEOMETRY ..... Starting Verification
  VERIFY GEOMETRY ..... Initializing
  VERIFY GEOMETRY ..... Deleting Existing Violations
  VERIFY GEOMETRY ..... Creating Sub-Areas
                  ..... bin size: 8000
  VERIFY GEOMETRY ..... SubArea : 1 of 1
  VERIFY GEOMETRY ..... Cells          :  0 Viols.
  VERIFY GEOMETRY ..... SameNet        :  0 Viols.
  VERIFY GEOMETRY ..... Wiring         :  0 Viols.
  VERIFY GEOMETRY ..... Antenna        :  0 Viols.
  VERIFY GEOMETRY ..... Sub-Area : 1 complete 0 Viols. 0 Wrngs.
VG: elapsed time: 1.00
Begin Summary ...
  Cells       : 0
  SameNet     : 0
  Wiring      : 0
  Antenna     : 0
  Short       : 0
  Overlap     : 0
End Summary

  Verification Complete : 0 Viols.  0 Wrngs.

***********End: VERIFY GEOMETRY***********
 *** verify geometry (CPU: 0:00:00.8  MEM: 35.5M)
```

```
encounter 1>
******** Start: VERIFY CONNECTIVITY ********
Start Time: Sun Oct 18 20:24:03 2026

Design Name: SYS_TOP
Database Units: 2000
Design Boundary: (0.0000, 0.0000) (240.4700, 160.4700)
Error Limit = 1000; Warning Limit = 50
Check all nets
Net RST_N: Found a geometry with bounding box (-0.10,70.10) (0.10,70.30) outside design boundary.
Violations for such geometries will be reported.

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Sun Oct 18 20:24:03 2026
******** End: VERIFY CONNECTIVITY ********
  Verification Complete : 0 Viols.  0 Wrngs.
  (CPU Time: 0:00:00.1  MEM: 0.000M)
```